第九章

建模与消隐

本章学习目标

- 柏拉图多面体的数据结构
- 光滑物体网格划分的数学模型
- 凸多面体的背面剔除算法
- 曲面的背面剔除算法
- 深度缓冲器算法

本章内容

9.1 三维物体的数据结构

9.2 消隐算法分类

9.3 隐线算法

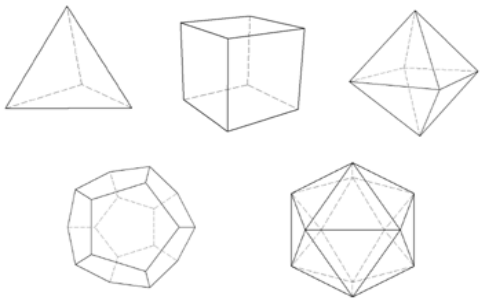
9.4 隐面算法

9.5 本章小结

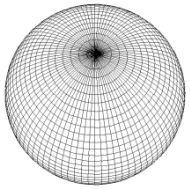
在二维显示器上绘制三维图形时，必须把三维信息经过投影变换为二维信息。由于投影变换失去了图形的深度信息，往往导致对图形的理解存在二义性。要生成具有真实感的图形，就要在给定视点和视线方向之后，决定场景中物体哪些线段或表面是可见的，哪些线段或表面是不可见的。这一问题习惯上称为消除隐藏线和消除隐藏面，简称为**消隐**。

9.1 三维物体的数据结构

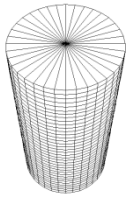
场景中经常绘制的三维物体有



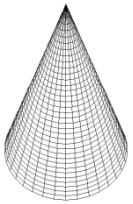
柏拉图多面体
Platonic Polyhedra



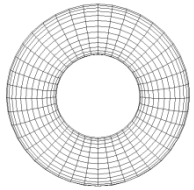
球
sphere



圆柱
cylinder



圆锥
cone



圆环
torus

这些物体可以采用线框模型描述，也可以采用表面模型或实体模型描述。无论使用哪种模型描述，都需要为物体建立顶点表、边表和面表构成的数据结构。建立三维用户坐标系为右手系Oxyz，x轴水平向右为正，y轴垂直向上为正，z轴从纸面指向观察者。

9.1.1 物体的几何信息和拓扑信息

几何信息：描述几何元素空间位置的信息。
拓扑信息：描述几何元素之间相互连接关系的信息。

描述一个物体不仅需要几何信息的描述而且需要拓扑信息的描述。因为只有几何信息的描述，在表示上存在不惟一性。图9-1所示的5个顶点，其几何信息已经确定，如果拓扑信息不同，则可产生图9-2和9-3所示的两种不同图形。

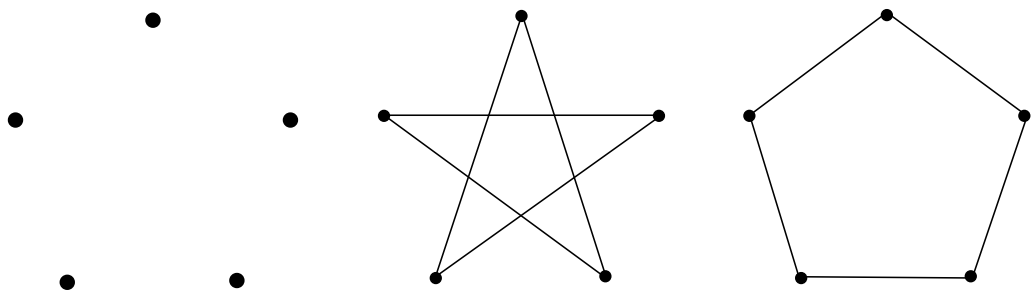


图9-1 五个顶点　图9-2 五角星连线　图9-3 五边形连线

对物体线框信息的描述不仅包括顶点坐标，而且包括每条边是由哪些顶点连接而成。对于物体表面信息的描述包括每个表面是由哪些边连接而成，或是由哪些顶点环绕而成。

9.1.2 三维物体的数据结构

在三维坐标系下，描述一个物体不仅需要顶点表描述其几何信息，而且还需要借助于边表和面表描述其拓扑信息，才能完全确定物体的几何形状。

制作多面体或曲面体的旋转动画时，常将物体的中心假设为旋转中心。假定立方体中心位于三维坐标系原点，立方体的边与坐标轴平行，且每条边的长度为2a。立方体模型如图9-4所示。立方体是凸多面体，满足欧拉公式：V+F-E=2式中：V是多面体的顶点数，F是多面体的面数，E是多面体的边数

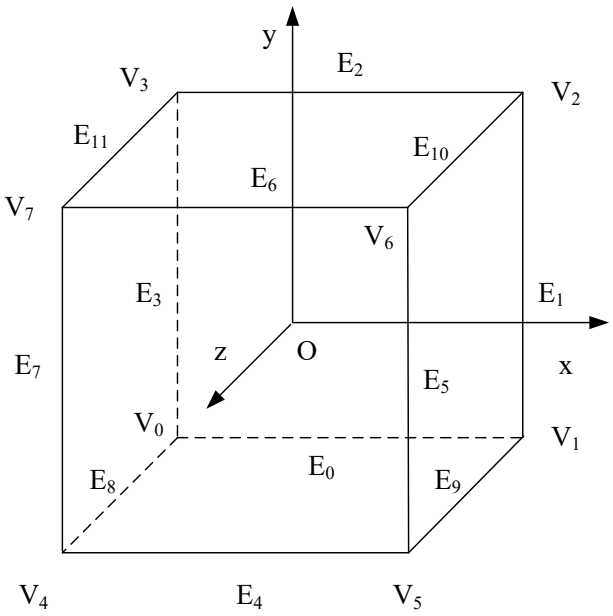


图9-4 立方体数学模型

表9-1 立方体顶点表

顶点	x坐标	y坐标	z坐标
V_0	$x_0=-a$	$y_0=-a$	$z_0=-a$
V_1	$x_1=a$	$y_1=-a$	$z_1=-a$
V_2	$x_2=a$	$y_2=a$	$z_2=-a$
V_3	$x_3=-a$	$y_3=a$	$z_3=-a$
V_4	$x_4=-a$	$y_4=-a$	$z_4=a$
V_5	$x_5=a$	$y_5=-a$	$z_5=a$
V_6	$x_6=a$	$y_6=a$	$z_6=a$
V_7	$x_7=-a$	$y_7=a$	$z_7=a$

表9-2 立方体边表

边	起点	终点
E_0	V_0	V_1
E_1	V_1	V_2
E_2	V_2	V_3
E_3	V_3	V_0
E_4	V_4	V_5
E_5	V_5	V_6
E_6	V_6	V_7
E_7	V_7	V_4
E_8	V_0	V_4
E_9	V_1	V_5
E_{10}	V_2	V_6
E_{11}	V_3	V_7

表9-3立方体面表

面	第1条边	第2条边	第3条边	第4条边	说明
F_0	E_4	E_5	E_6	E_7	前面
F_1	E_0	E_3	E_2	E_1	后面
F_2	E_3	E_8	E_7	E_{11}	左面
F_3	E_1	E_{10}	E_5	E_9	右面
F_4	E_2	E_{11}	E_6	E_{10}	顶面
F_5	E_0	E_9	E_4	E_8	底面

9.1.3 实体的描述模型

1. 线框模型

线框模型（wireframe model）是计算机图形学中表示物体最早使用的模型，而且一直在使用。线框模型只是用几何体的边线来表示物体的外形，没有表面和体积等概念。线框模型是表面模型和实体模型的基础，只使用顶点表和边表两个数据结构就可以描述。图9-5所示为立方体线框模型。

优点:可以产生任意方向视图，视图间保持正确的投影关系，常用于绘制三视图或斜轴测图等。

缺点：所有棱边全部绘制出来，容易产生二义性,如图9-6所示

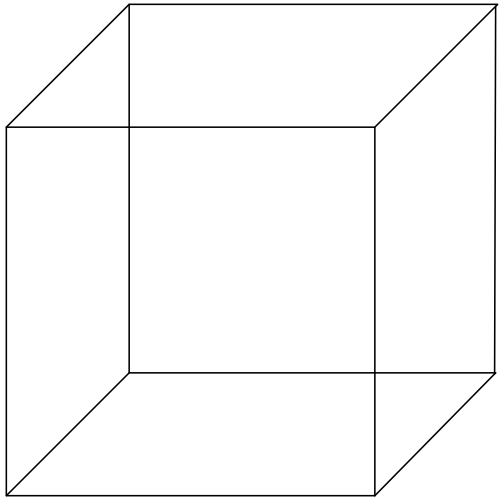


图9-5立方体线框模型

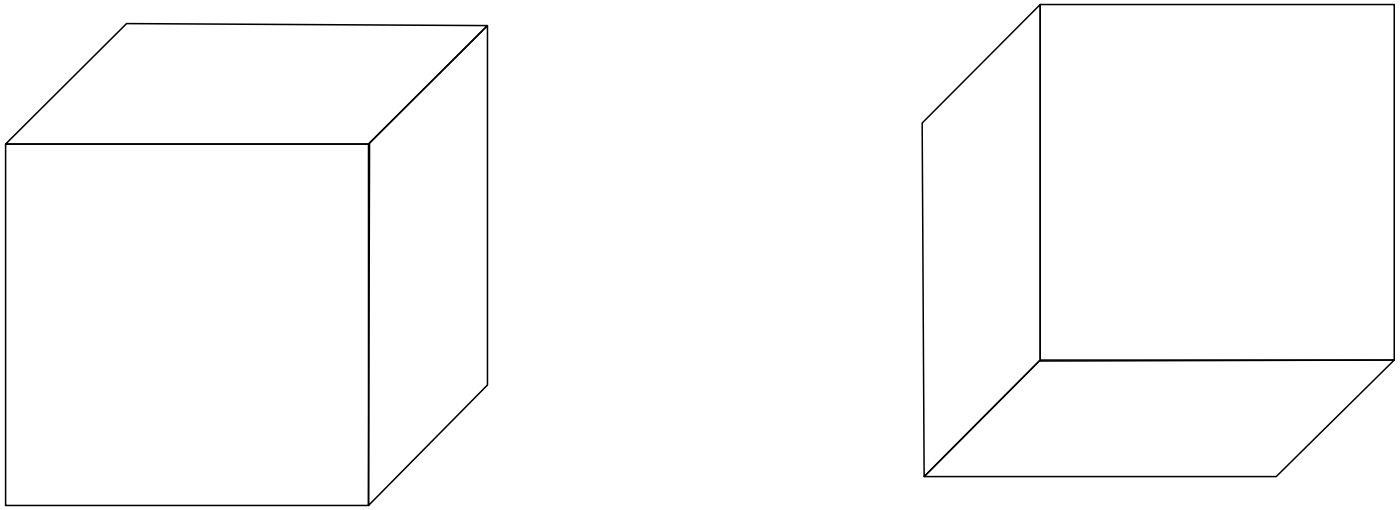


图9-6线框模型二义性

2. 表面模型

表面模型（surface model）是利用物体的外表面来构造模型，就如同在线框模型上蒙上了一层外皮，使物体具有了一层外表。表面模型仍缺乏体积的概念，是一个物体的空壳。与线框模型相比，表面模型增加了一个面表，用以记录边面之间的拓扑关系。

优点：可以对表面进行平面着色或光滑着色、可以为物体添加光照或纹理等。

缺点：无法进行实体之间的并交叉运算。图9-7表示的是双三次Bezier曲面的网格模型。图9-8表示的是双三次Bezier曲面的表面模型。在图9-8中，Bezier曲面没有围成一个封闭的空间，只是一张很薄的面片，其表面无内外之分，哪面是正面、哪面是反面，没有给出明确的定义。

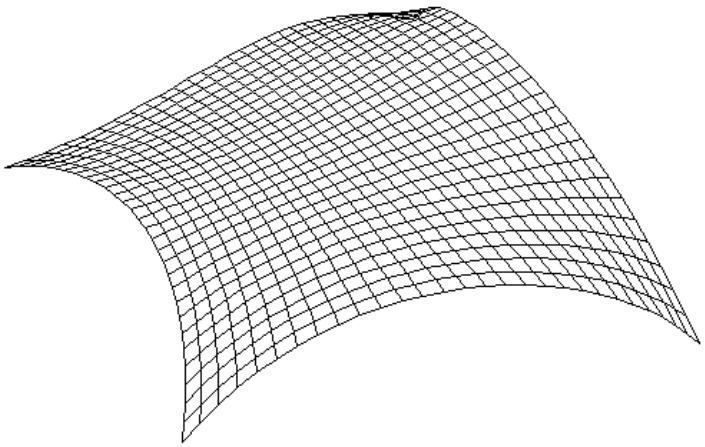


图9-7 双三次Bezier线框模型

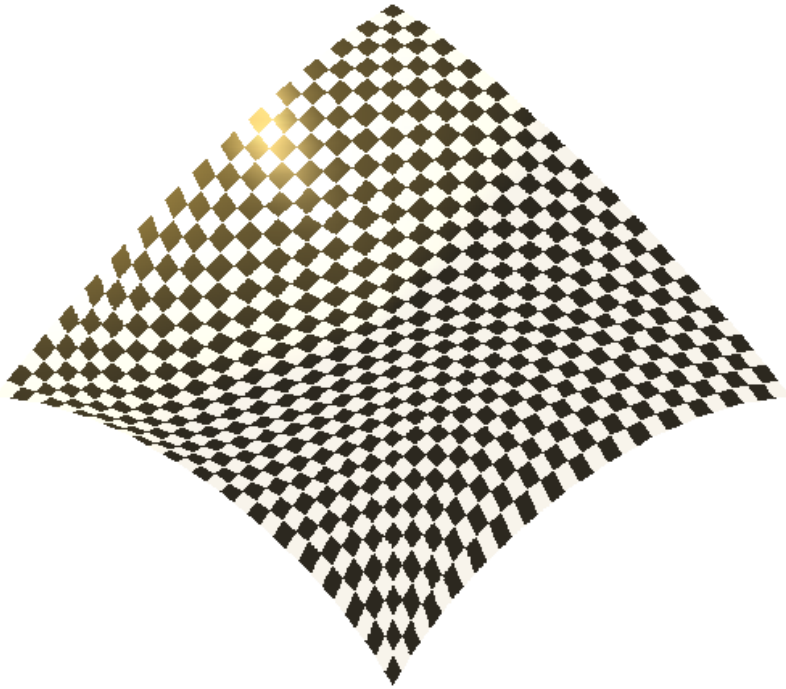


图9-8双三次Bezier表面模型

3.实体模型

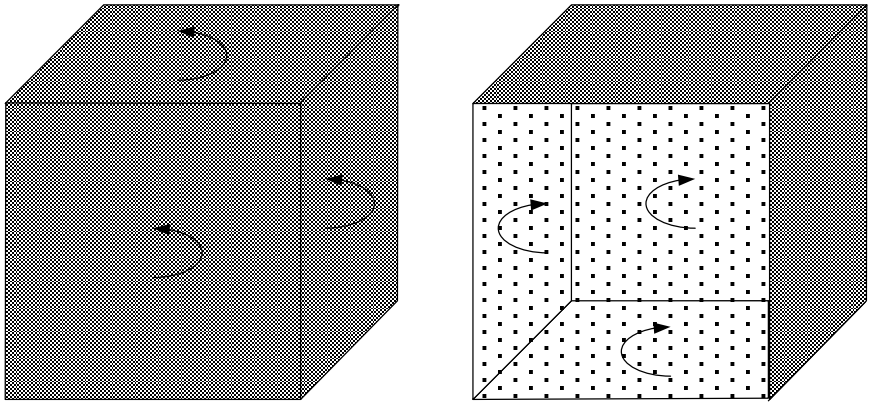
实体模型（solid model)是在封闭的表面模型内部进行了填充，有了如体积和重量等特性，能反映立体的真实性，立体才具有“体”的概念。它的内部和外部的概念，定义了在表面模型的哪一侧存在实体。它的表面有正面和反面之分。如图9-9所示。

用有向棱边隐含地表示表面的外法向量方向。常用右手准则定义，拓扑合法的物体在相邻两个面的公共边界上，棱边的方向正好相反，如图9-10所示。

与表面模型数据结构的差异。将面表的顶点索引号按照从物体外部观察的逆时针方向的顺序排列，可确切地分清体内体外。

与线框模型、表面模型的区别。记录了顶点的信息，以及线、面、体的拓扑信息。

实体模型常采用集合论中的并、交、差等运算来构造复杂实体 。



（a）正面　　（b）反面

图9-9 立方体表面的正面和反面

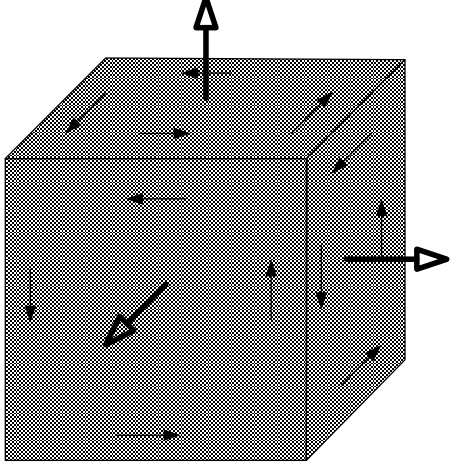


图9-10 立方体实体模型

一般情况下，使用顶点表、边表和面表3张表可以方便地检索到物体的任意一个顶点、任意一条边和任意一个表面，而且数据结构清晰。

实际建模中，实体模型采用了有向棱边，相邻两个表面上共享的一条棱边的定义方向截然相反，导致无法确定棱边的顶点连接顺序，因而放弃边表，仅使用顶点表和面表来表示物体的几何模型。且面表中按照表面法矢量向外的方向遍历多边形顶点索引号，表明处理的是物体的正面。仅用顶点表和面表的缺点是物体的每条边被重复绘制2次。

9.1.4 双表结构

不管是三维凸多面体还是光滑物体，只要给出顶点表和面表数据文件，就可以正确地确定数据结构。在双表结构中，立方体的顶点表依然使用表9-1。面表需要重新按顶点索引号设计。图9-11为图9-4所示立方体的展开图。

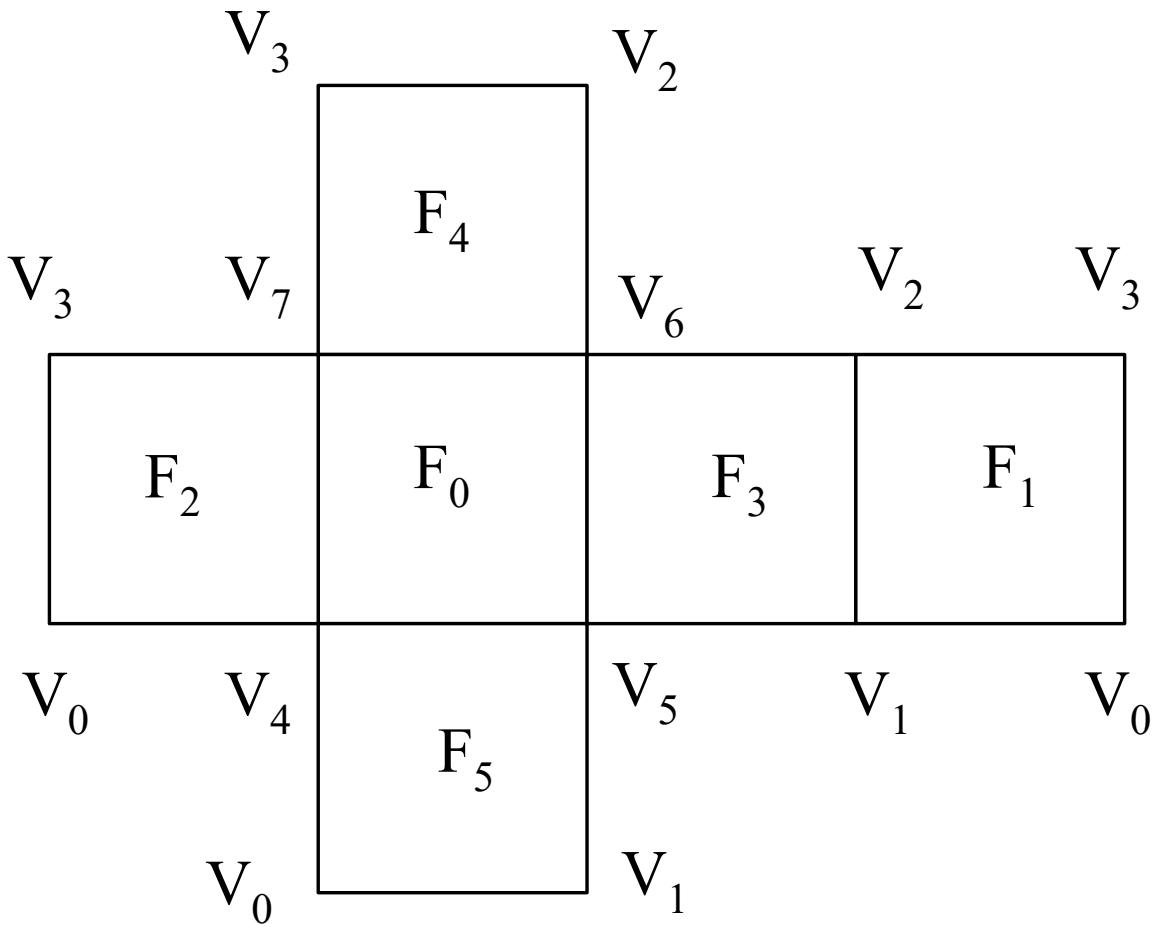


图9-11 立方体的展开图

表9-4根据立方体的展开图重新设计了面表结构。

表9-4立方体面表

面	第一个顶点	第二个顶点	第三个顶点	第四个顶点	说明
F_0	4	5	6	7	前面
F_1	0	3	2	1	后面
F_2	0	4	7	3	左面
F_3	1	2	6	5	右面
F_4	2	3	7	6	顶面
F_5	0	1	5	4	底面

1．定义三维顶点类

如图9-12所示，包括顶点的三维坐标（x,y,z)

```
class CP3
{
public:
        CP3();
        virtual ~CP3();
        CP3(double,double,double);
public:
        double x;
        double y;
        double z;
};
```

图9-12 三维顶点类

2．定义表面类

如图9-13,表面类包括表面的顶点数和表面的顶点索引号。SetNum()用于动态设置表面的顶点数，常用于处理3个顶点的三角形面片或4个顶点的四边形面片。

```
class CFace
{
public:
        CFace();
        virtual ~CFace();
        void SetNum(int); //设置面片的顶点数
public:
        int vN;             //面片的顶点数
        int *vI;            //面片的顶点索引
};
```

图9-13 表面类

3．读入立方体的点表

在程序中定义ReadVertex()函数读入物体的顶点表如图9-14

```
void CTestView::ReadVertex()//读入顶点表
{
        double a=100;//立方体的半边长
        //顶点的三维坐标(x,y,z)
        V[0].x=-a;V[0].y=-a;V[0].z=-a;
        V[1].x=+a;V[1].y=-a;V[1].z=-a;
        V[2].x=+a;V[2].y=+a;V[2].z=-a;
        V[3].x=-a;V[3].y=+a;V[3].z=-a;
        V[4].x=-a;V[4].y=-a;V[4].z=+a;
        V[5].x=+a;V[5].y=-a;V[5].z=+a;
        V[6].x=+a;V[6].y=+a;V[6].z=+a;
        V[7].x=-a;V[7].y=+a;V[7].z=+a;
}
```

图9-14 读入立方体的顶点表

4．读入立方体的面表

在程序中定义ReadFace()函数读入物体的面表，如图9-15

```
void CTestView::ReadFace()
{
        //面的顶点数和面的顶点索引号
        F[0].SetNum(4);F[0].vI[0]=4;F[0].vI[1]=5;F[0].vI[2]=6;F[0].vI[3]=7;//前面
        F[1].SetNum(4);F[1].vI[0]=0;F[1].vI[1]=3;F[1].vI[2]=2;F[1].vI[3]=1;//后面
        F[2].SetNum(4);F[2].vI[0]=0;F[2].vI[1]=4;F[2].vI[2]=7;F[2].vI[3]=3;//左面
        F[3].SetNum(4);F[3].vI[0]=1;F[3].vI[1]=2;F[3].vI[2]=6;F[3].vI[3]=5;//右面
        F[4].SetNum(4);F[4].vI[0]=2;F[4].vI[1]=3;F[4].vI[2]=7;F[4].vI[3]=6;//顶面
        F[5].SetNum(4);F[5].vI[0]=0;F[5].vI[1]=1;F[5].vI[2]=5;F[5].vI[3]=4;//底面
}
```

图9-15 读入立方体的面表

9.1.5 常用物体的数学模型

1. 柏拉图多面体

正多面体是由若干个全等的正多边形围成，并且相交在各个顶点上的棱边数都相等的凸多面体。正多面体只有正四面体、正六面体、正八面体、正十二面体和正二十面体五种，如图9-16 所示。表9-5给出了其几何信息。这五种多面体统称为柏拉图多面体。柏拉图多面体属于凸多面体。

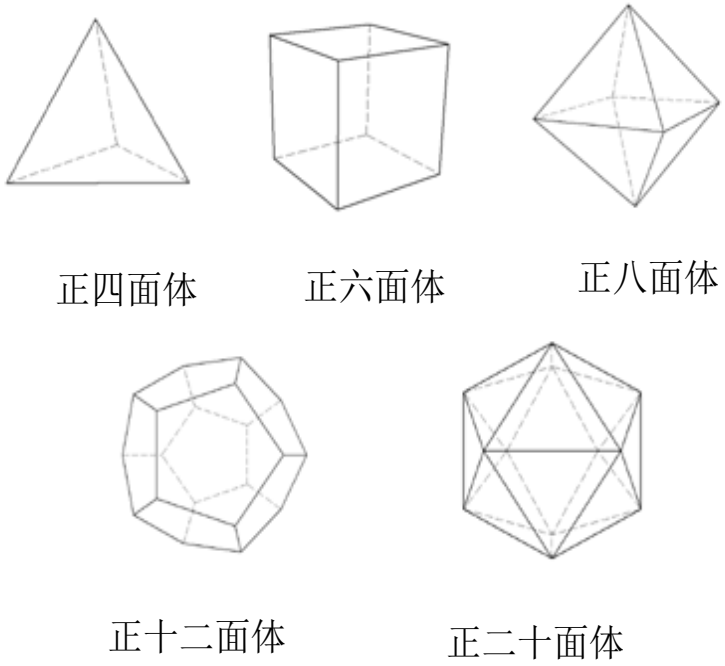


图9-16 柏拉图多面体

在几何学中，若一种多面体的每个顶点均能对应到另一种多面体上的每个面的中心，二者互称为对偶多面体。

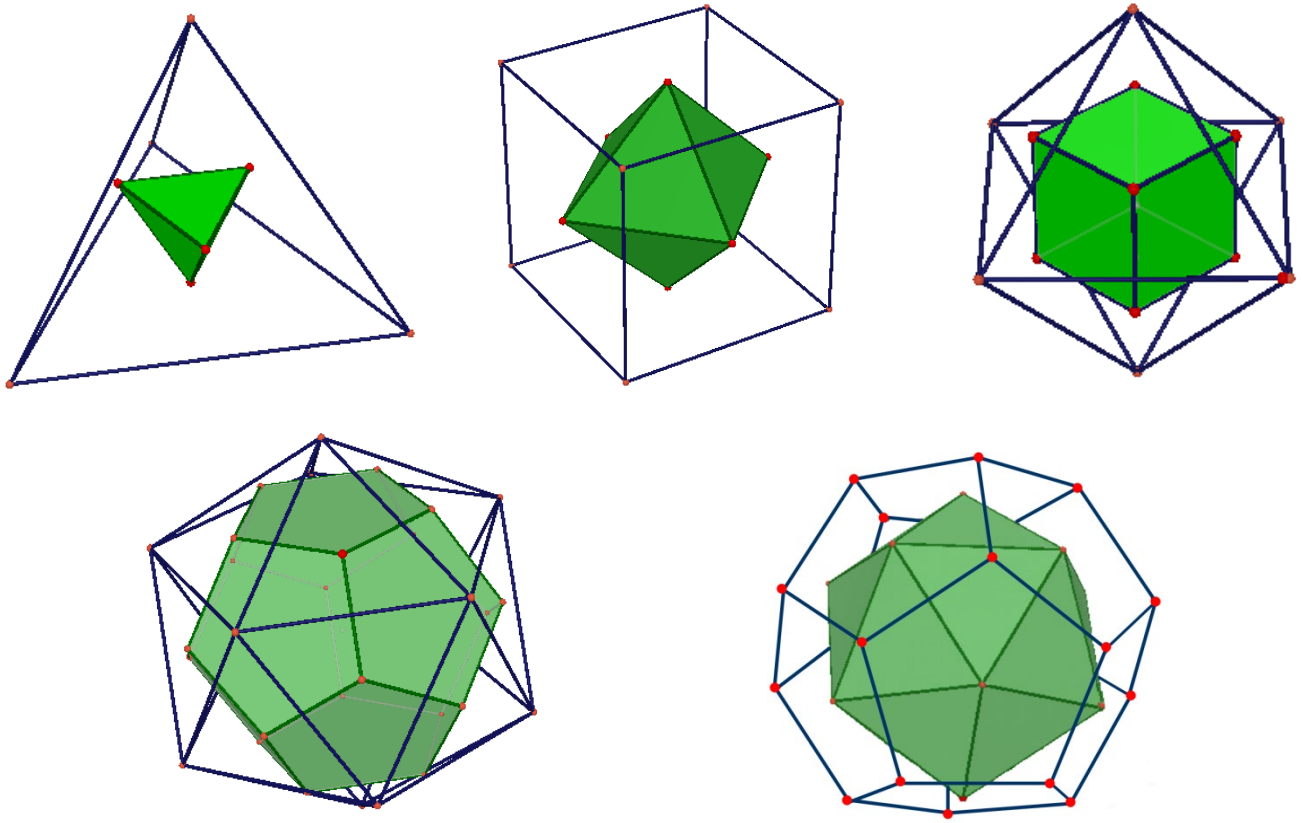


图9-17 柏拉图多面体对偶多面体

柏拉图多面体对偶性

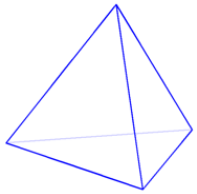
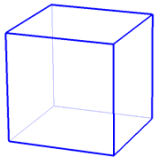
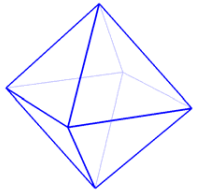
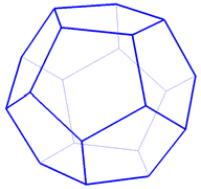
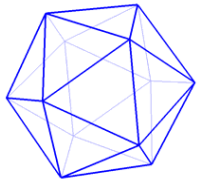
					
点	4	8	6	20	12
线	6	12	12	30	30
面	4	6	8	12	20

表9-5 柏拉图多面体几何信息统计

多面体	正四面体	正六面体	正八面体	正十二面体	正二十面体
顶点数V	4	8	6	20	12
边数E	6	12	12	30	30
面数F	4	6	8	12	20
面的形状	正三角形	正方形	正三角形	正五边形	正三角形

〔1〕正四面体

建立正四面体的伴随立方体可以很容易地确定正四面体的顶点表和面表。

正四面体的外接球和其伴随立方体的外接球是同一个球；正四面体外接球的直径就是立方体的对角线。假设立方体的半边长为a，令V_0点为（a，a，a），则正V_1 V_2 V_3顶点表见表9-6。

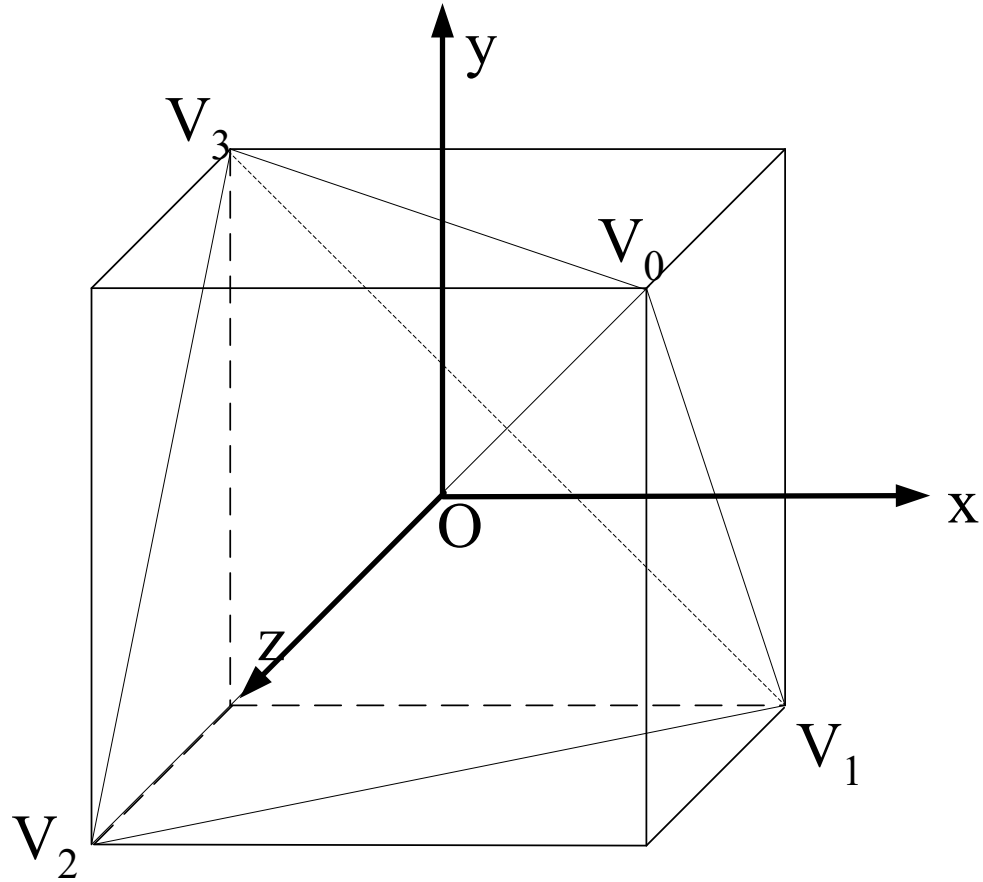


图9-18 正四面体几何模型

表9-6 正四面体顶点表

顶点	x坐标	y坐标	z坐标
V_0	$x_0 = a$	$y_0 = a$	$z_0 = a$
V_1	$x_1 = a$	$y_1 = -a$	$z_1 = -a$
V_2	$x_2 = -a$	$y_2 = -a$	$z_2 = a$
V_3	$x_3 = -a$	$y_3 = a$	$z_3 = -a$

表9-7 正四面体面表

面	第一个顶点	第二个顶点	第三个顶点
F_0	1	2	3
F_1	0	3	2
F_2	0	1	3
F_3	0	2	1

正四面体的外接球面的半径为 $r=\sqrt{3}a$展开图如图9-19所示，面表见表9-7。

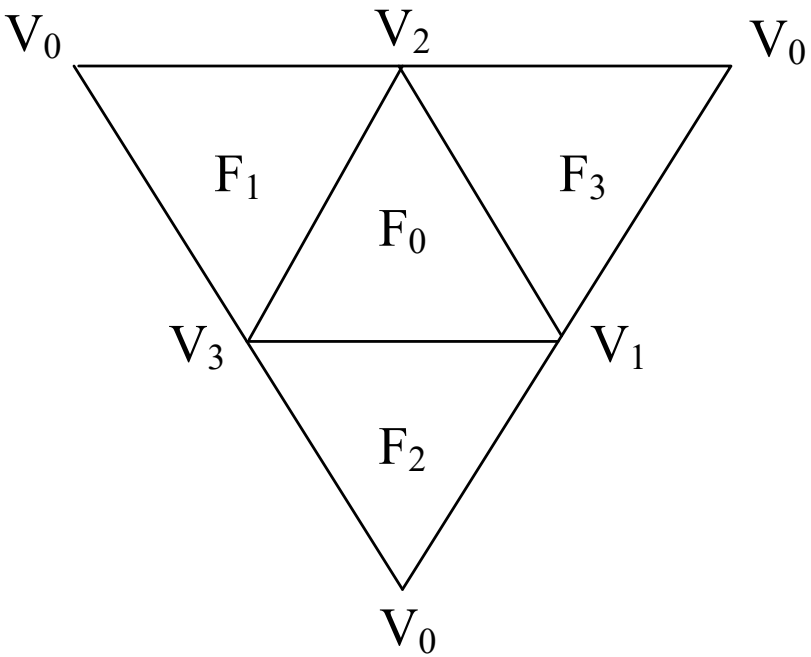
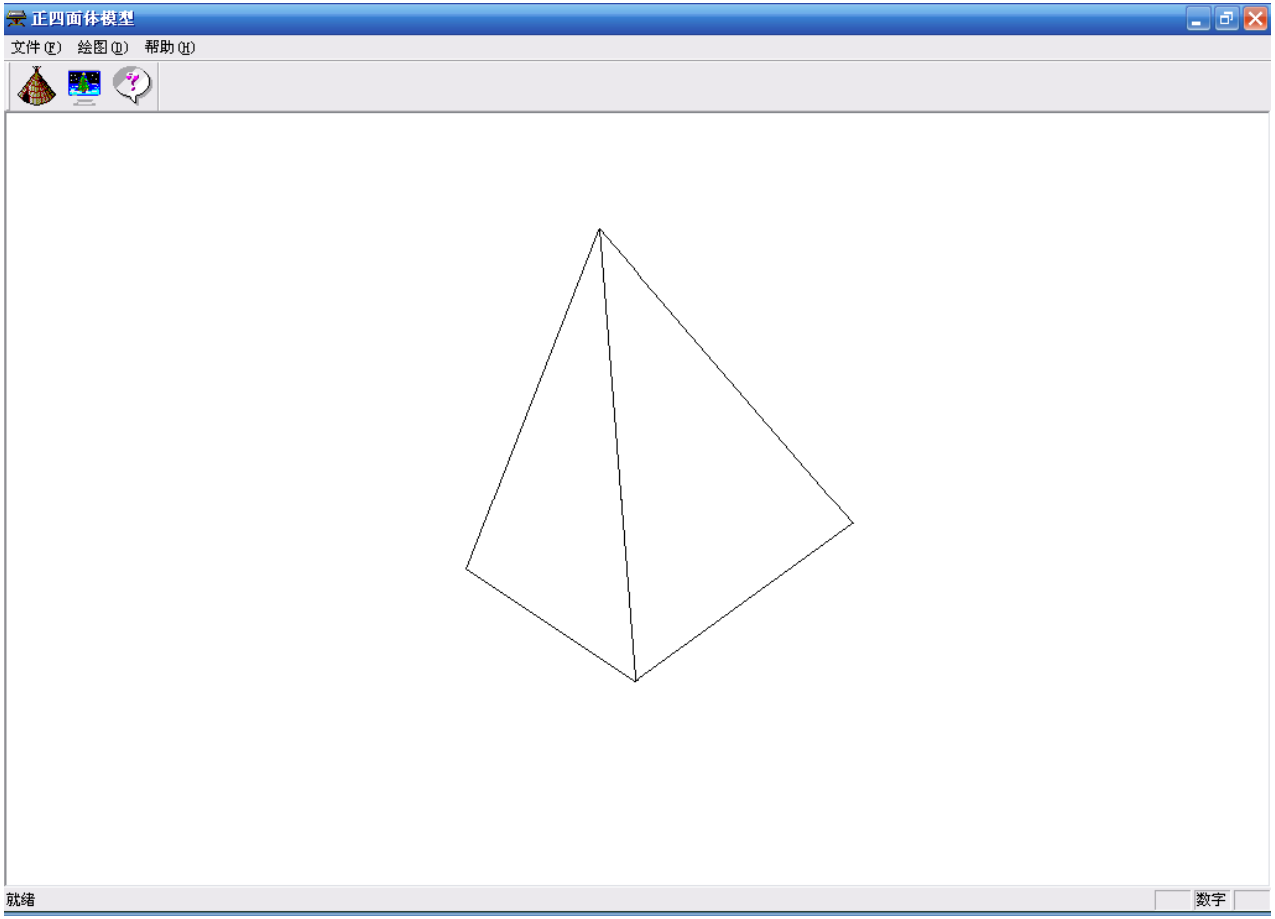


图9-19 正四面体展开图



正四面体

〔2〕正八面体

设正八面体的外接球面的半径为r，6个顶点都取自坐标轴，并且两两关于原点对称，如图9-20所示，顶点表见表9-8。根据图9-21所示的正八面体展开图可以得到正八面体的面表，见表9-9。

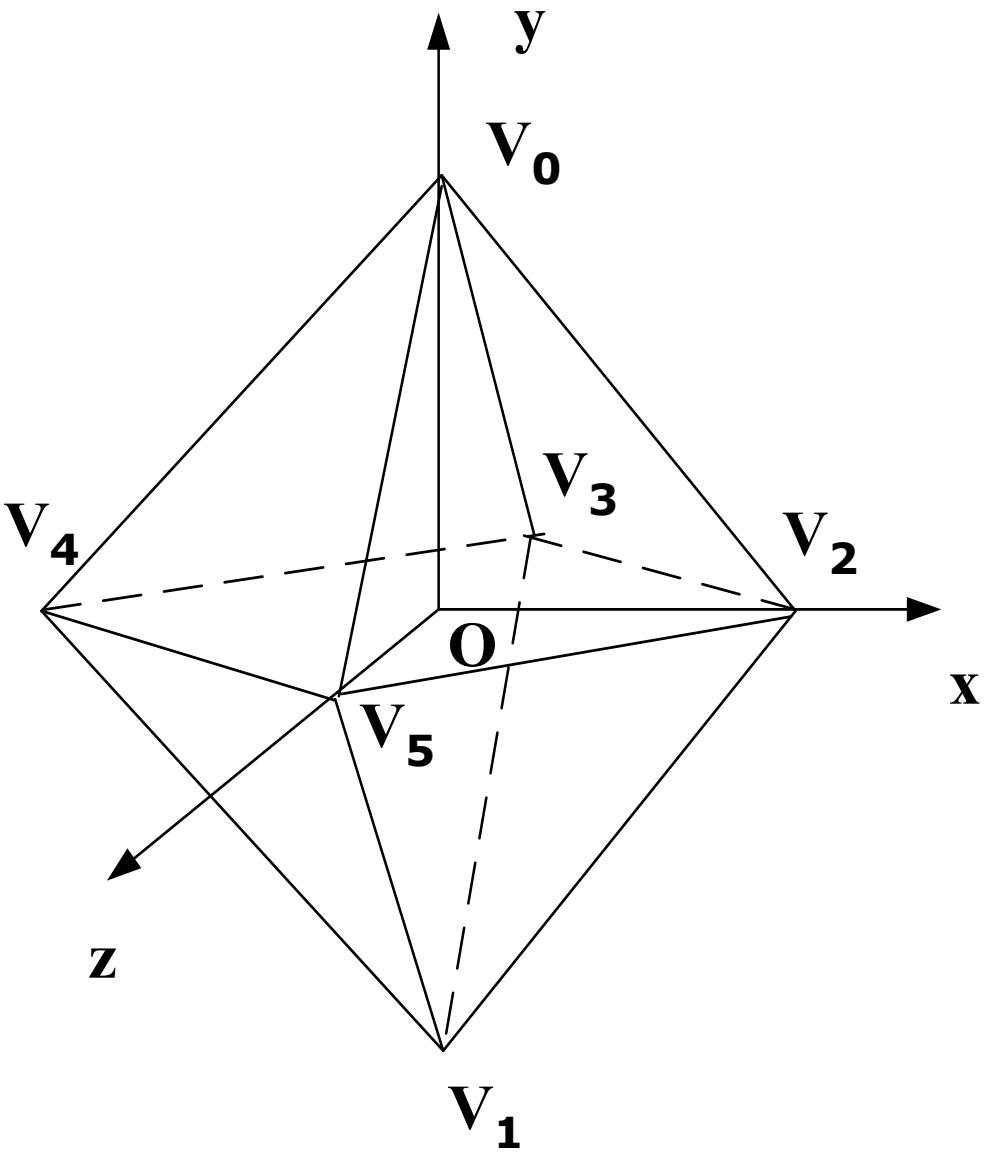
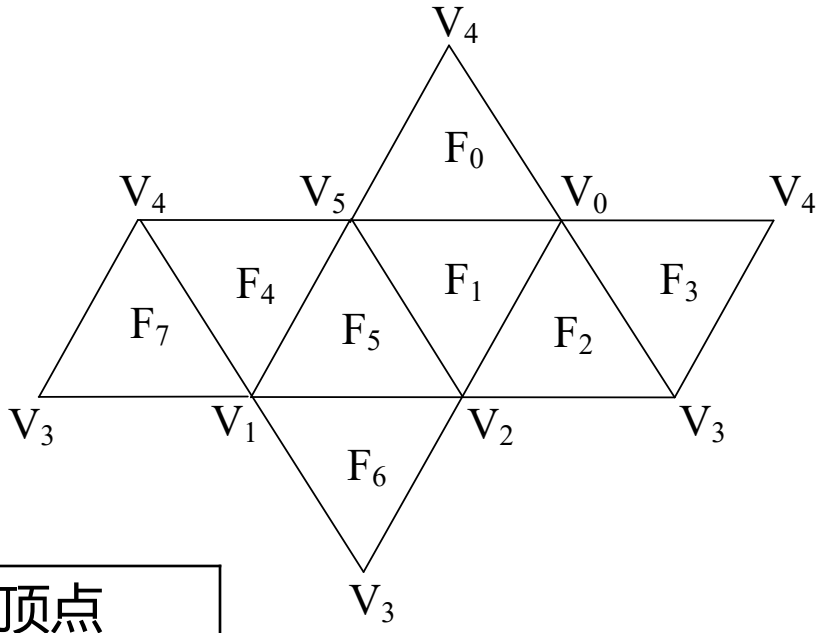


图9-20 正八面体几何模型

表9-8 正八面体顶点表

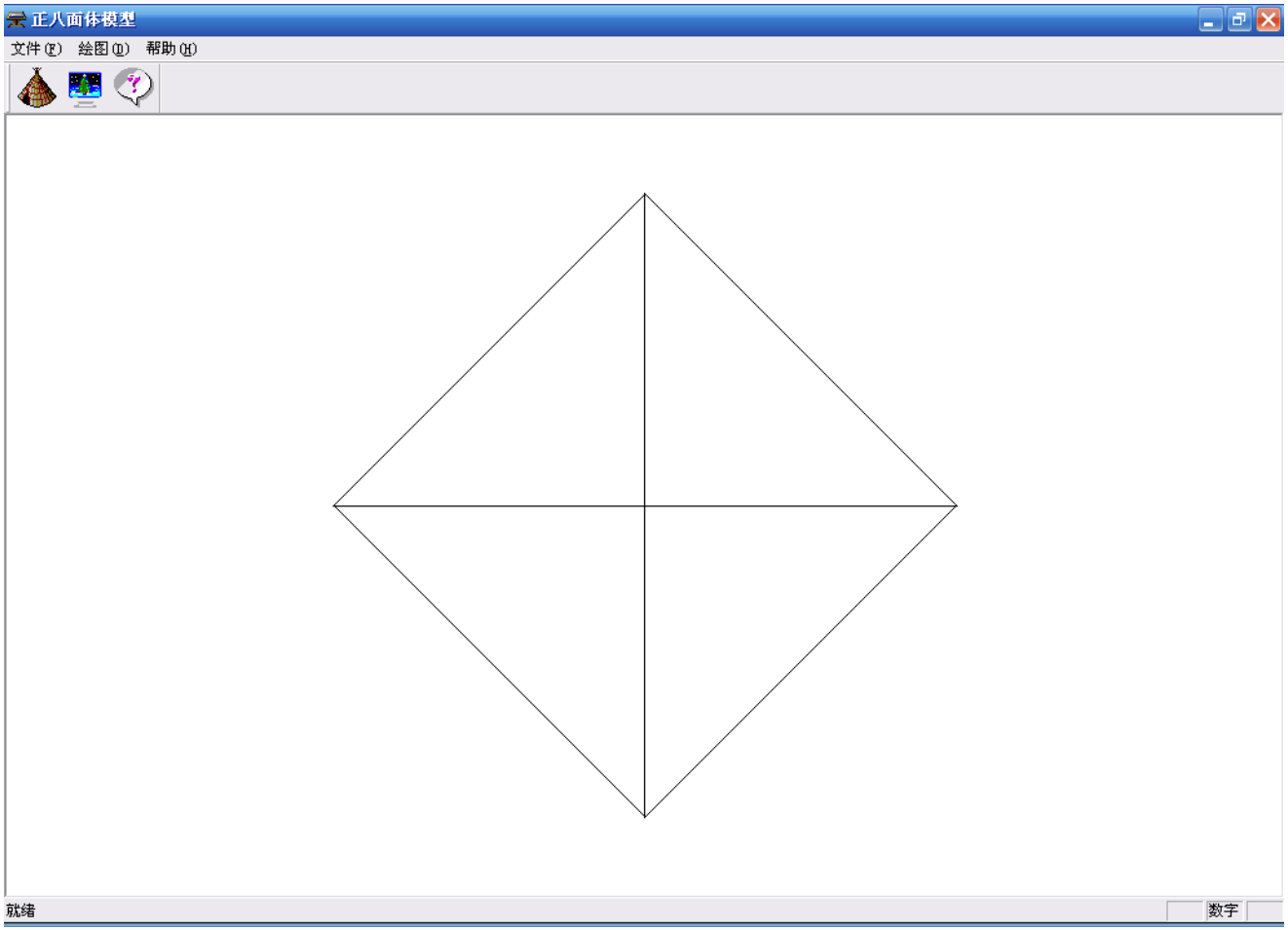
顶点	x坐标	y坐标	z坐标
V_0	$x_0 = 0$	$y_0 = r$	$z_0 = 0$
V_1	$x_1 = 0$	$y_1 = -r$	$z_1 = 0$
V_2	$x_2 = r$	$y_2 = 0$	$z_2 = 0$
V_3	$x_3 = 0$	$y_3 = 0$	$z_3 = -r$
V_4	$x_4 = -r$	$y_4 = 0$	$z_4 = 0$
V_5	$x_5 = 0$	$y_3 = 0$	$z_5 = r$

图9-21正八面体展开图



面	第一个顶点	第二个顶点	第三个顶点
F_0	0	4	5
F_1	0	5	2
F_2	0	2	3
F_3	0	3	4
F_4	1	5	4
F_5	1	2	5
F_6	1	3	2
F_7	1	4	3

表9-9 正八面体面表



正八面体模型

〔3〕正十二面体

建立正十二面体的坐标系如图9-22所示。三个互相垂直的矩形是黄金矩形。

黄金矩形就是矩形的短边与长边之比为：

$$\varphi = (\sqrt{5} - 1)/2 = 0.618$$

φ被称为黄金分割数，简称黄金数。

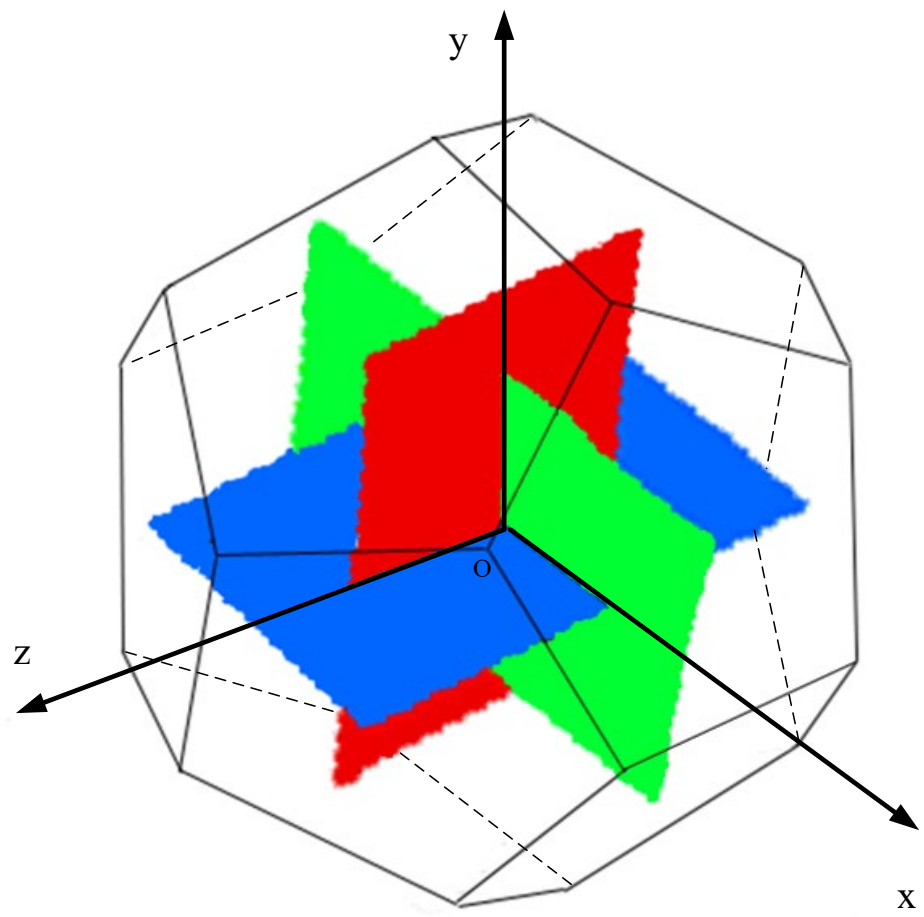
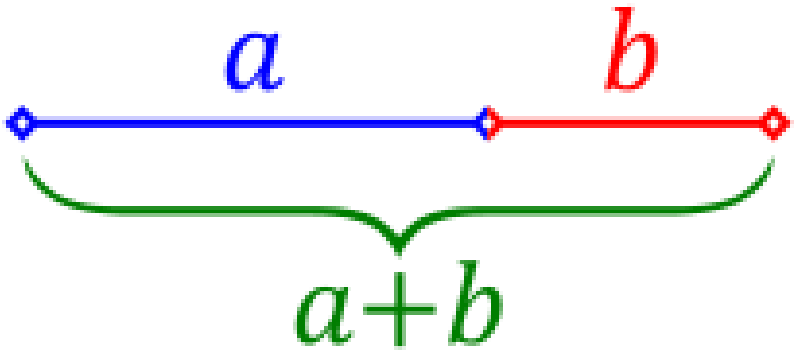


图9-22 正十二面体坐标系

黄金数

把一条线段分割为两部分，使其中一部分与全长之比等于另一部分与这部分之比。其比值是一个无理数，取其前三位数字的近似值是0.618。由于按此比例设计的造型十分美丽，0.618被称为黄金分割数。

$$\frac{b}{a}=\frac{a}{a+b}=\varphi$$



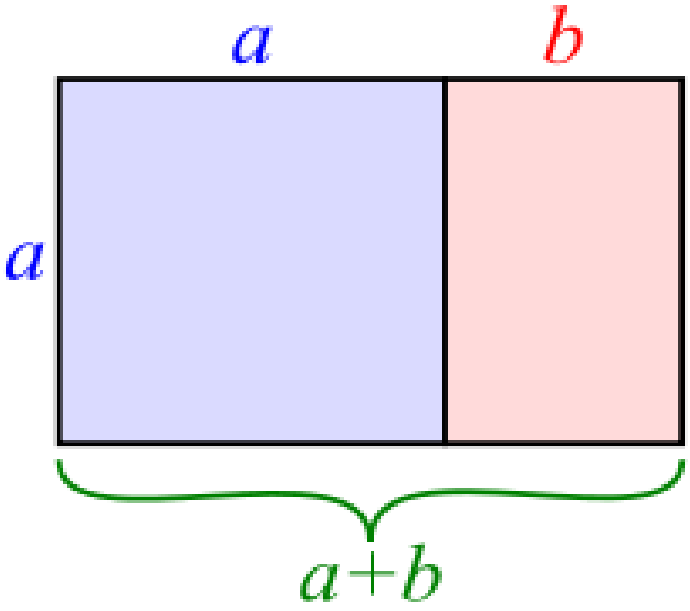
推导: $\varphi=\dfrac{1}{1+\varphi}$ $\quad \varphi^2+\varphi=1$ $\quad \varphi^2+\varphi+\dfrac{1}{4}=1+\dfrac{1}{4}$

$$(\varphi+\frac{1}{2})^2=\frac{5}{4} \qquad \varphi=\frac{\sqrt{5}-1}{4}=0.618$$

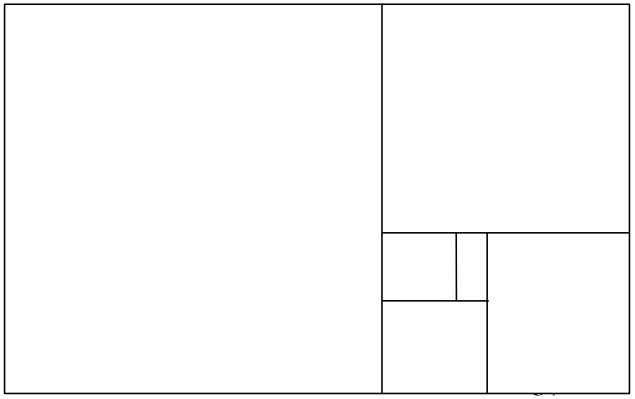
$$\frac{1}{\varphi}=\frac{\sqrt{5}+1}{2}=1.6180339$$,即 $$\frac{1}{\varphi}=1+\varphi$$

黄金矩形

$$\frac{a}{a+b}=\frac{b}{a}=\varphi$$

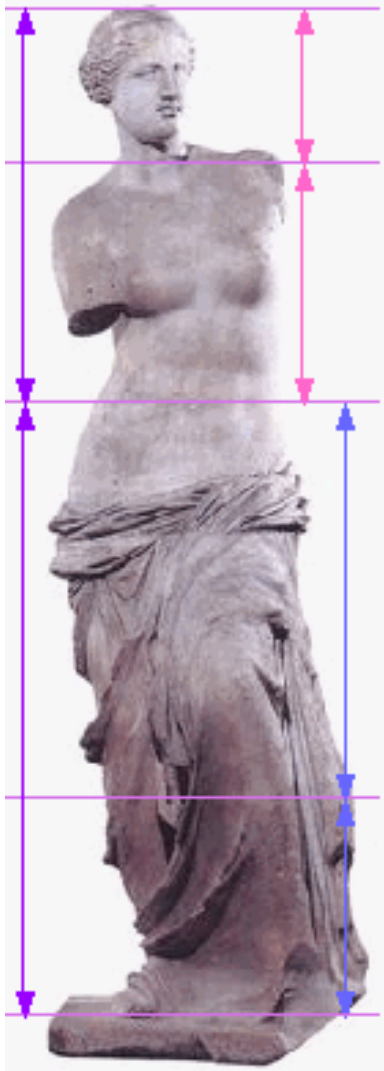


黄金矩形分割为两块,其中一块为正方形,另一块为黄金为矩形，可以递归划分。

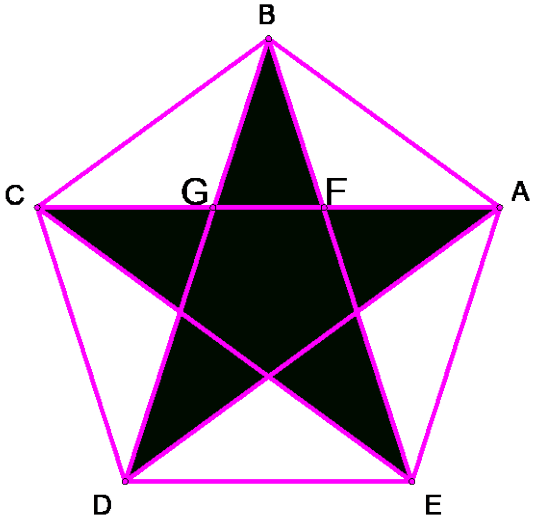


黄金比例

- 维纳斯雕像,被视为女性体型美的标准。
- (1)人的肚跻位于人体身高的黄金分割点。
- (2)肚跻以上,颈部是黄金分割点。
- (3)肚跻以下,膝盖是黄金分割点。



黄金三角形



点F为线段CA黄金分割点。
点G为线段CF黄金分割点。

黄金矩形的顶角位于十二面体的十二个表面中心。设黄金矩形的短边半边长为b，则黄金矩形的长边半边长为$a = b/\varphi$。表9-10为正十二面体的顶点表。

表9-10 正十二面体顶点表

顶点	x坐标	y坐标	z坐标
V_0	$x_0 = a$	$y_0 = a$	$z_0 = a$
V_1	$x_1 = a+b$	$y_1 = 0$	$z_1 = b$
V_2	$x_2 = a$	$y_2 = -a$	$z_2 = a$
V_3	$x_3 = 0$	$y_3 = -b$	$z_3 = a+b$
V_4	$x_4 = 0$	$y_4 = b$	$z_4 = a+b$
V_5	$x_5 = a+b$	$y_5 = 0$	$z_5 = -b$
V_6	$x_6 = a$	$y_6 = a$	$z_6 = -a$
V_7	$x_7 = b$	$y_7 = a+b$	$z_7 = 0$
V_8	$x_8 = -b$	$y_8 = a+b$	$z_8 = 0$
V_9	$x_9 = -a$	$y_9 = a$	$z_9 = -a$

接下页

顶点	x坐标	y坐标	z坐标
V_{10}	$x_{10} = 0$	$y_{10} = b$	$z_{10} = -a-b$
V_{11}	$x_{11} = a$	$y_{11} = -a$	$z_{11} = -a$
V_{12}	$x_{12} = b$	$y_{12} = -a-b$	$z_{12} = 0$
V_{13}	$x_{13} = -b$	$y_{13} = -a-b$	$z_{13} = 0$
V_{14}	$x_{14} = -a-b$	$y_{14} = 0$	$z_{14} = b$
V_{15}	$x_{15} = -a$	$y_{15} = a$	$z_{15} = a$
V_{16}	$x_{16} = -a$	$y_{16} = -a$	$z_{16} = -a$
V_{17}	$x_{17} = 0$	$y_{17} = -b$	$z_{17} = -a-b$
V_{18}	$x_{18} = -a$	$y_{18} = -a$	$z_{18} = a$
V_{19}	$x_{19} = -a-b$	$y_{19} = 0$	$z_{19} = -b$

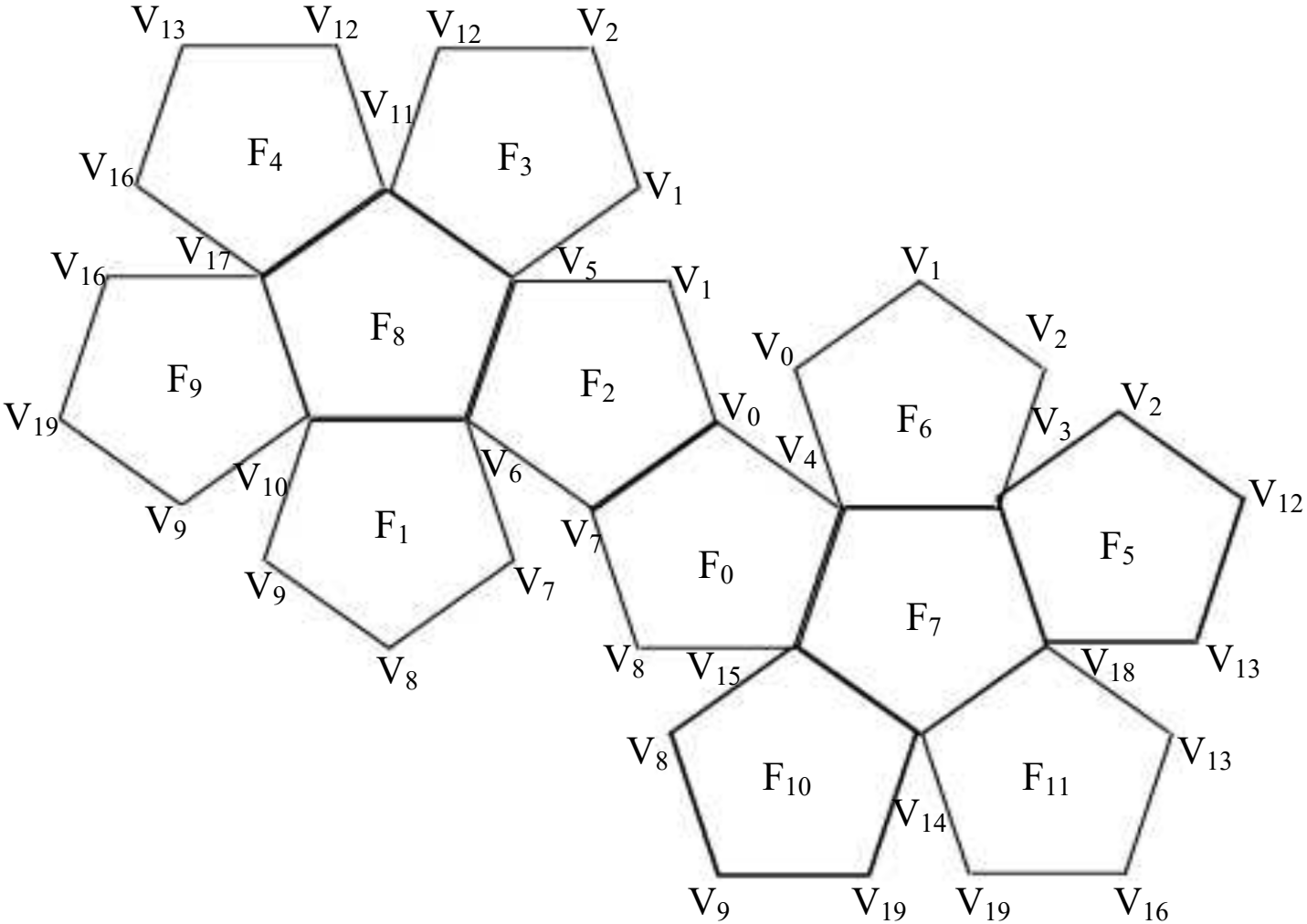
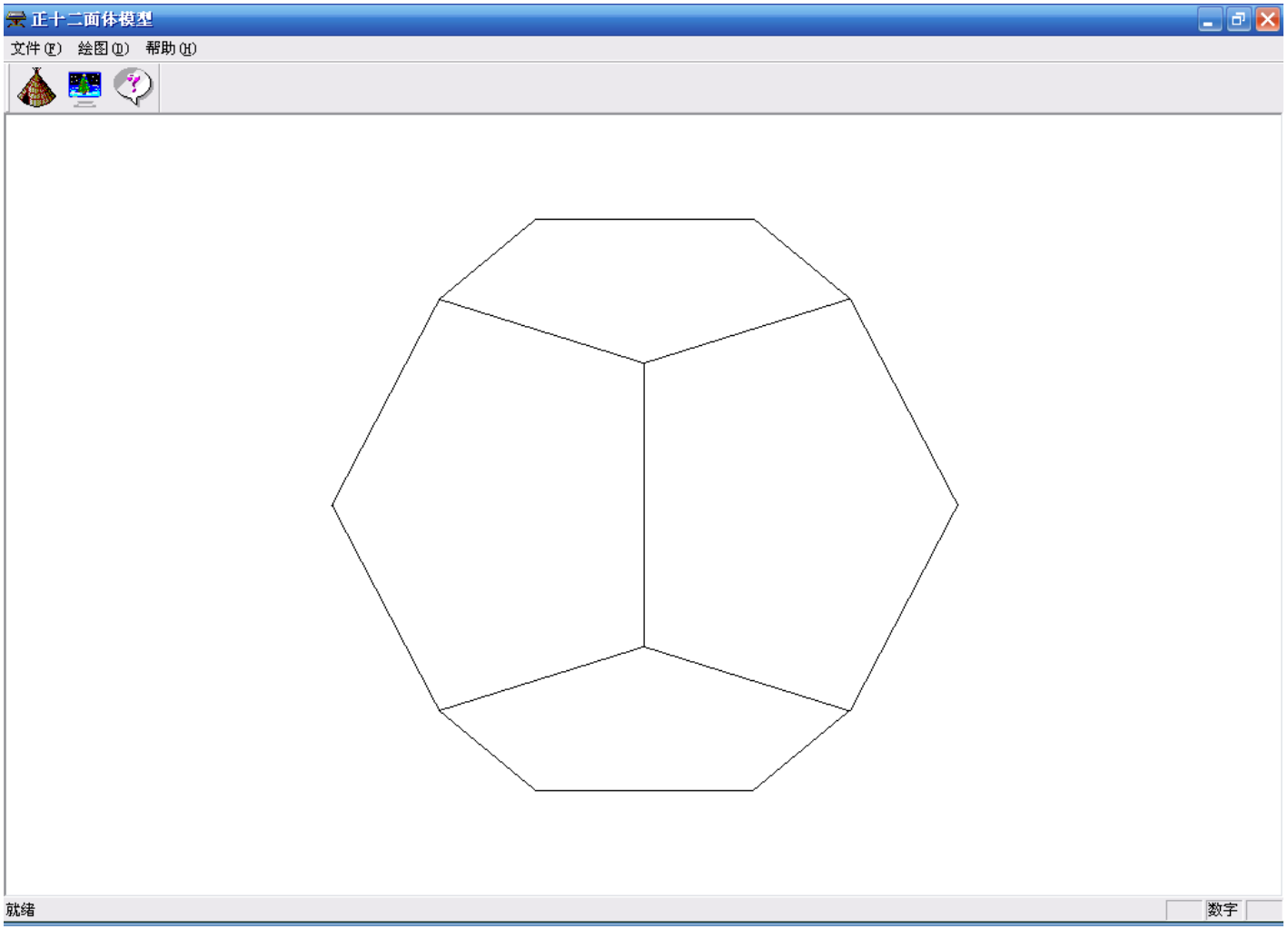


图9-23 正十二面体展开图

表9-11 正十二面体面表

面	第1顶点	第2顶点	第3顶点	第4顶点	第5顶点
F_0	0	7	8	15	4
F_1	6	10	9	8	7
F_2	1	5	6	7	0
F_3	1	2	12	11	5
F_4	11	12	13	16	17
F_5	2	3	18	13	12
F_6	0	4	3	2	1
F_7	3	4	15	14	18
F_8	5	11	17	10	6
F_9	9	10	17	16	19
F_{10}	8	9	19	14	15
F_{11}	13	18	14	19	16



正十二面体模型

〔4〕正二十面体

图9-24中三个黄金矩形两两正交，这些矩形的顶角是正二十面体的十二个顶点。把每一个黄金矩形与一个坐标轴对齐，得到表9-12的顶点表。由图9-25所示的正二十面体展开图得到正二十面体的面表，见表9-13。

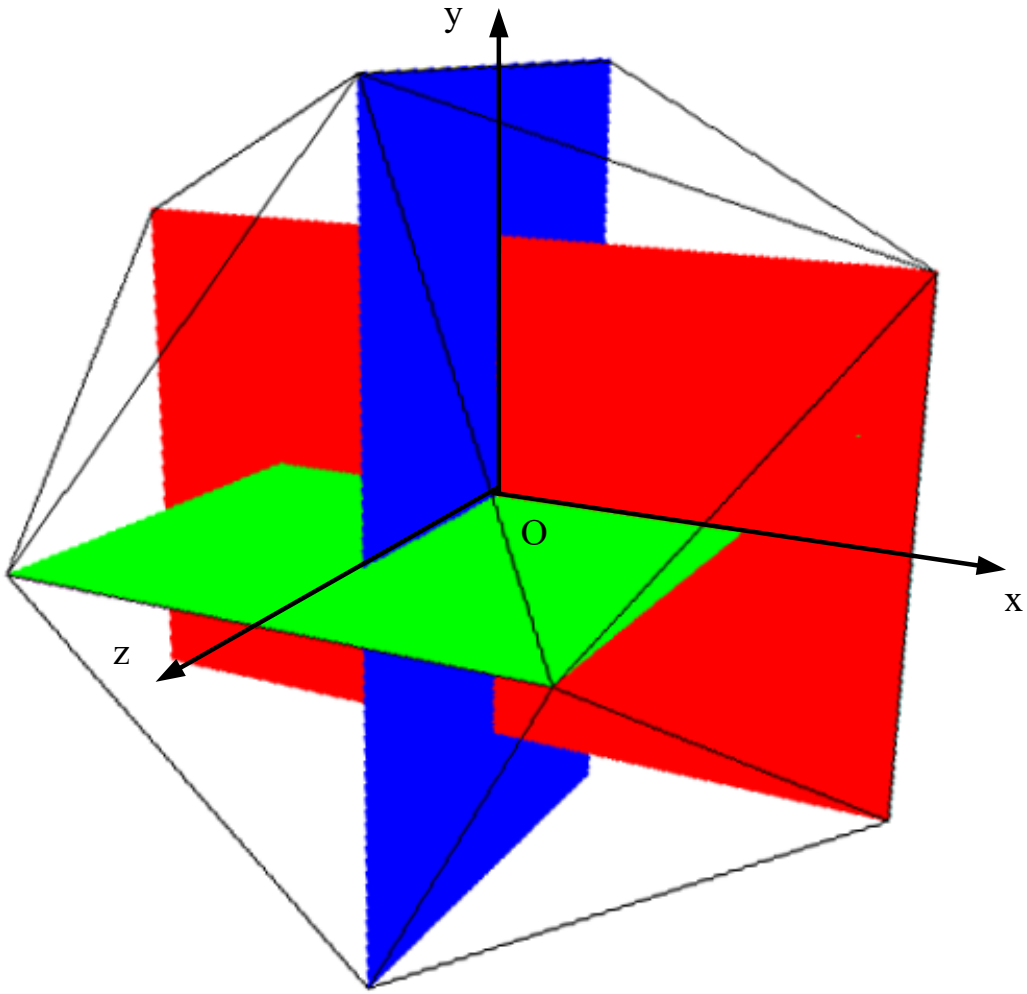


图9-24 用黄金矩形定义正二十面体

表9-12 正二十面体顶点表

顶点	x坐标	y坐标	z坐标
V_0	$x_0 = 0$	$y_0 = a$	$z_0 = b$
V_1	$x_1 = 0$	$y_1 = a$	$z_1 = -b$
V_2	$x_2 = a$	$y_2 = b$	$z_2 = 0$
V_3	$x_3 = a$	$y_3 = -b$	$z_3 = 0$
V_4	$x_4 = 0$	$y_4 = -a$	$z_4 = -b$
V_5	$x_5 = 0$	$y_3 = -a$	$z_5 = b$
V_6	$x_6 = b$	$y_6 = 0$	$z_6 = a$
V_7	$x_7 = -b$	$y_7 = 0$	$z_7 = a$
V_8	$x_8 = b$	$y_8 = 0$	$z_8 = -a$
V_9	$x_9 = -b$	$y_9 = 0$	$z_9 = -a$
V_{10}	$x_{10} = -a$	$y_{10} = b$	$z_{10} = 0$
V_{11}	$x_{11} = -a$	$y_{11} = -b$	$z_{11} = 0$

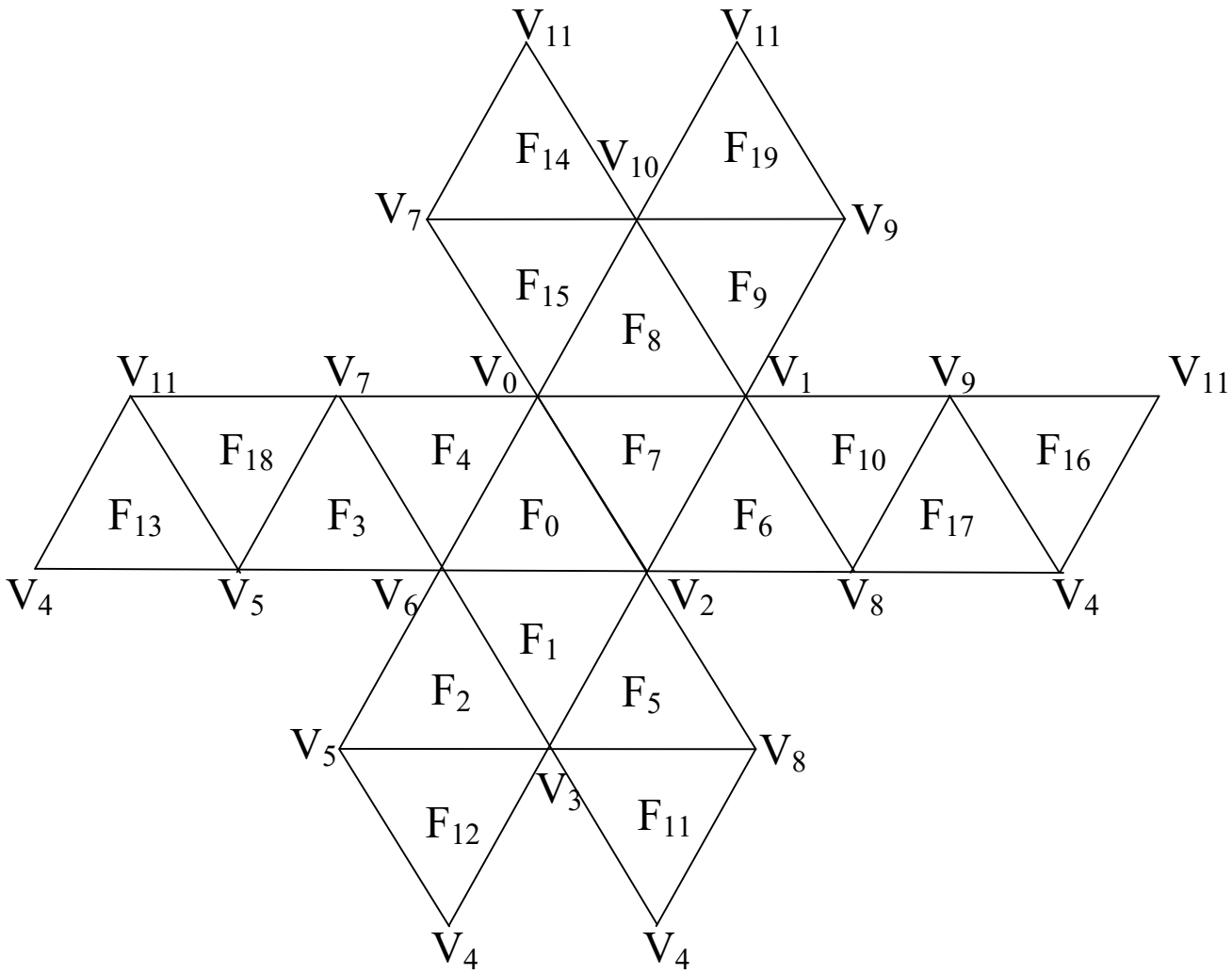


图9-25 正二十面体展开图

表9-13 正二十面体面表

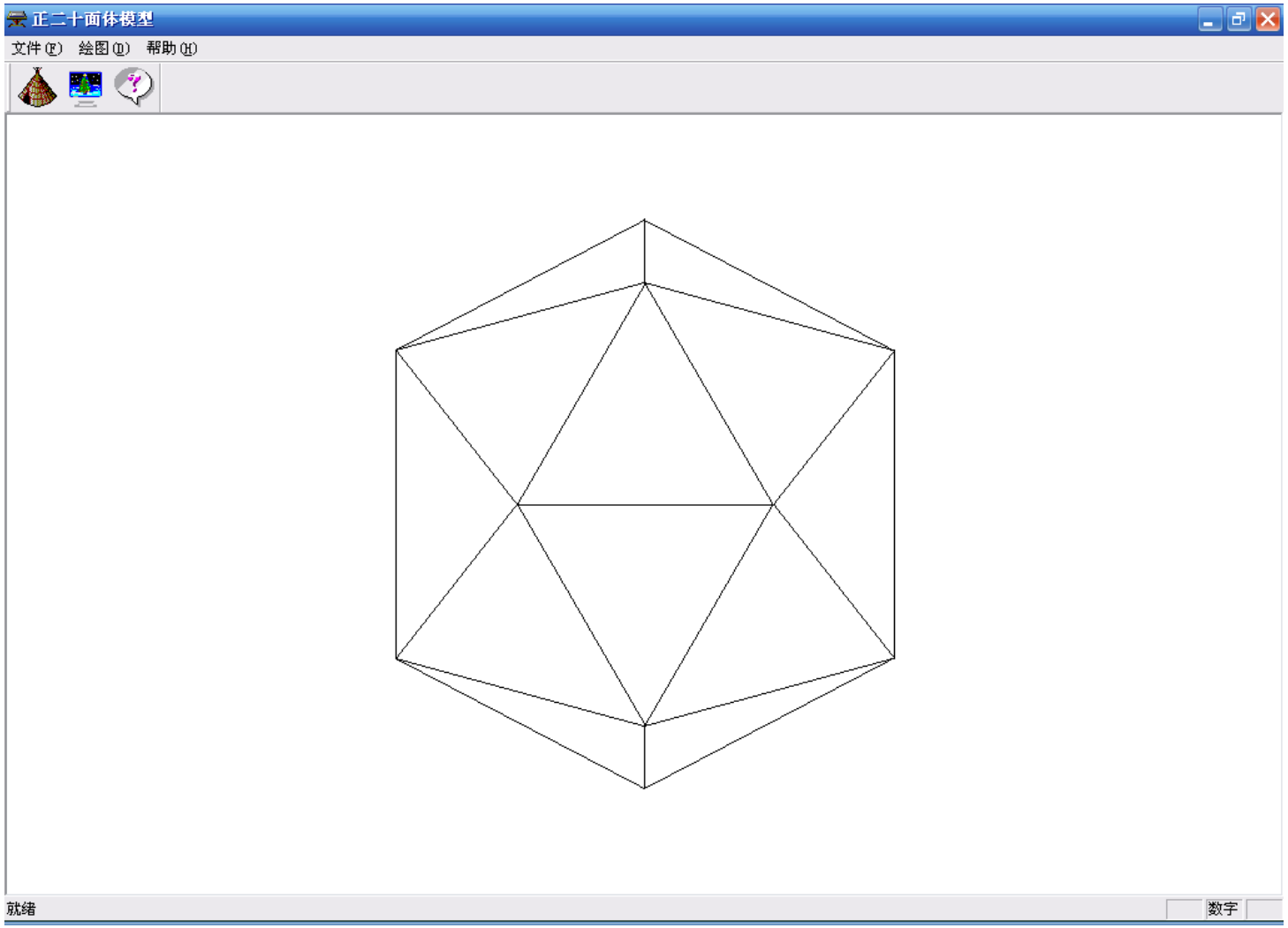
面	第一个顶点	第二个顶点	第三个顶点
F_0	0	6	2
F_1	2	6	3
F_2	3	6	5
F_3	5	6	7
F_4	0	7	6
F_5	2	3	8
F_6	1	2	8
F_7	0	2	1

表9-13 正二十面体面表

面	第一个顶点	第二个顶点	第三个顶点
F_8	0	1	0
F_9	1	9	10
F_{10}	1	8	9
F_{11}	3	4	8
F_{12}	3	5	4
F_{13}	4	5	11
F_{14}	7	10	11
F_{15}	0	10	7

表9-13 正二十面体面表

面	第一个顶点	第二个顶点	第三个顶点
F_{16}	4	11	9
F_{17}	4	9	8
F_{18}	5	7	11
F_{19}	9	11	10



正二十面体模型

2. 光滑物体

多面体是由平面多边形组成的物体，多面体没有方程表示形式，用顶点表和面表直接给出数据结构定义。

对于球、圆柱、圆锥、圆环等光滑物体，表面有确定的参数方程表示形式。绘制光滑物体时，需要进行网格划分，即把光滑曲面离散为平面多边形表示，这些多边形一般为平面四边形或三角形网格。光滑物体的网格顶点表和面表使用物体的参数方程离散计算后得到。

〔1〕球面

球心在原点，半径为r的球面三维坐标系如图9-26所示。球面的参数方程表示为

$$\begin{cases} x = r\sin\alpha\sin\beta \\ y = r\cos\alpha \\ z = r\sin\alpha\cos\beta \end{cases} \qquad (0 \le \alpha \le \pi, 0 \le \beta \le 2\pi) \qquad (9\text{-}2)$$

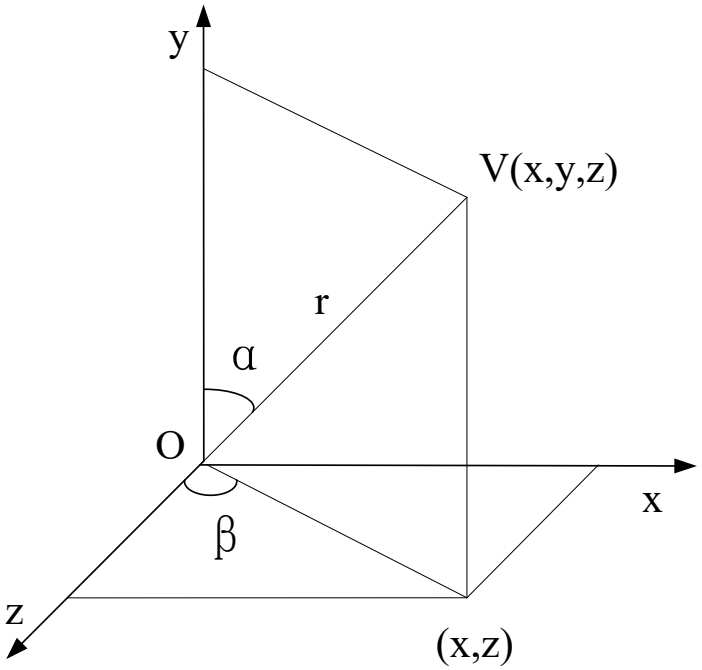


图9-26 球面的数学模型

假定将球面划分为$n_1 = 4$个纬度区域，$n_2 = 8$个经度区域。则纬度方向的角度增量和经度方向的角度增量均为$\alpha=\beta=45°$，示例球面的网格模型如图9-27所示。

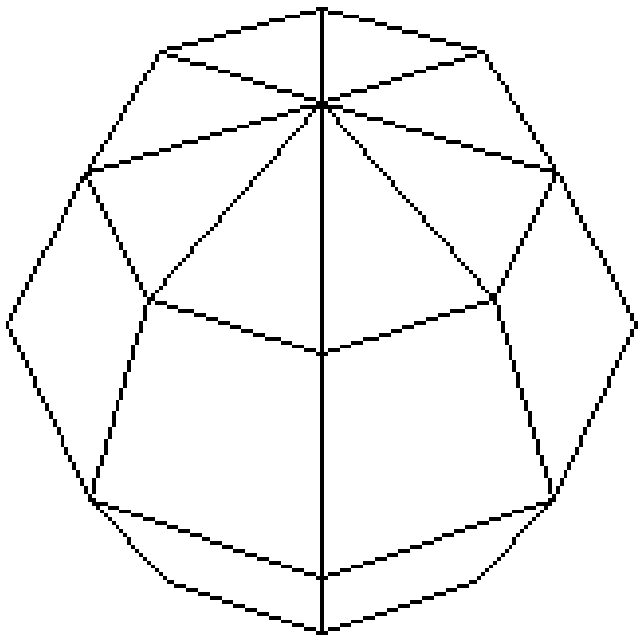


图9-27 示例球面

球面共有$(n_1-1)*n_2+2=26$个顶点。顶点索引号为0～25。北极点序号为0，然后从z轴正向开始，绕y轴按逆时针方向确定位于第一条纬度线上与各条经度线相交的点，如图9-28和图9-29所示，最后一个顶点为南极点。北极点坐标为V_0（0,r,0），南极点坐标为V_{25}（0, -r,0）。

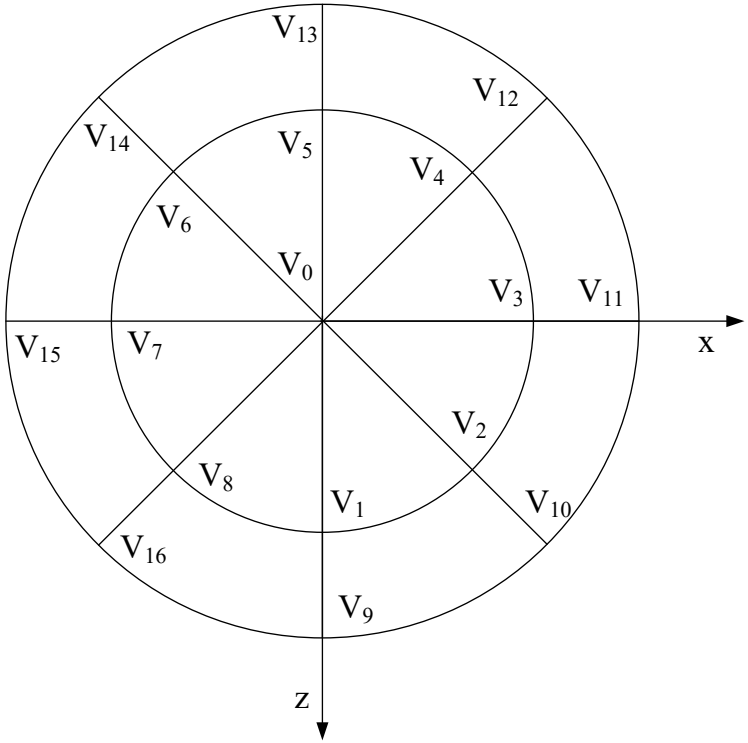


图9-28北半球顶点编号

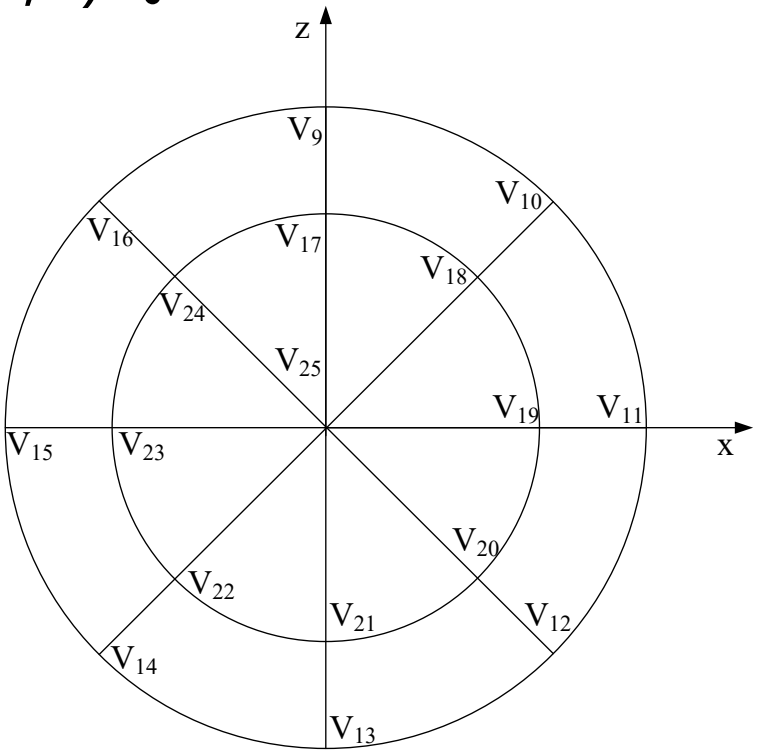


图9-29南半球顶点编号

面表用二维数组定义，第一维表示纬度自北极向南极递增的方向，第二维表示在同一纬度线上从z轴正向开始，绕y轴的逆时针方向。如图9-30和图9-31所示。所有网格的顶点排列顺序应以小面的法线指向球面外部的右手法则为准。

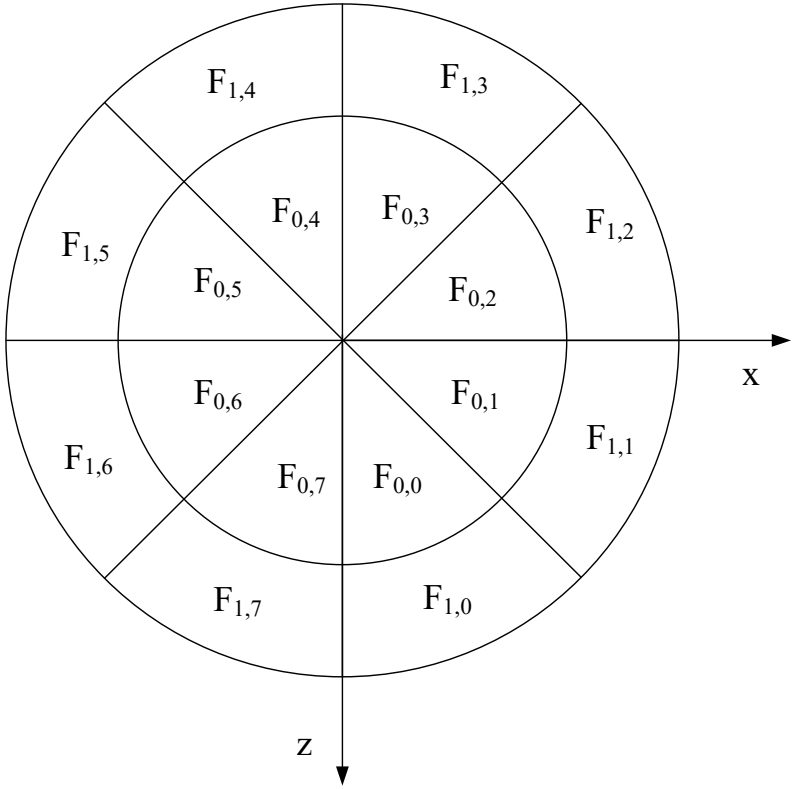


图9-30北半球表面编号

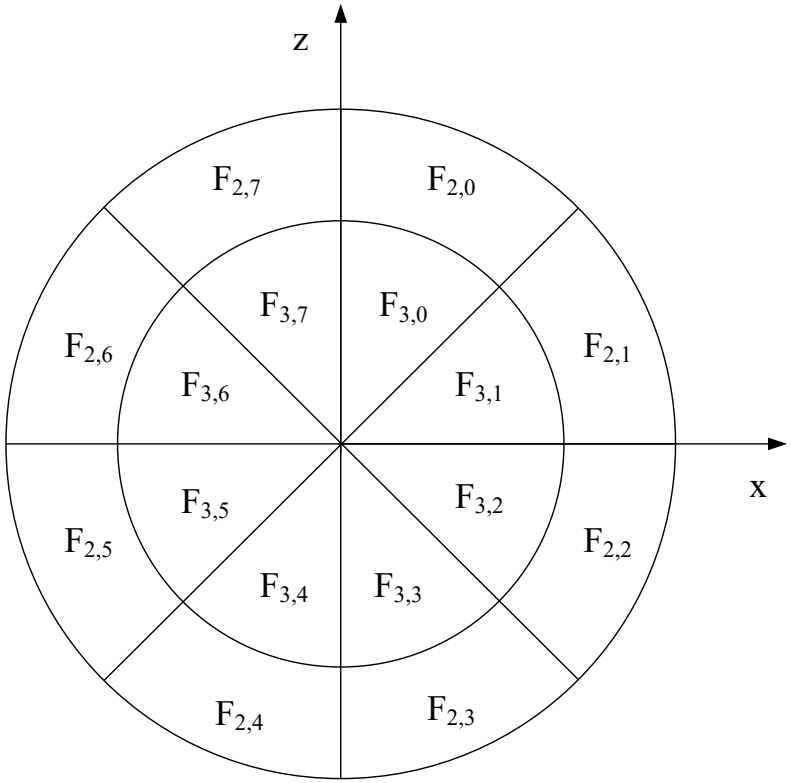
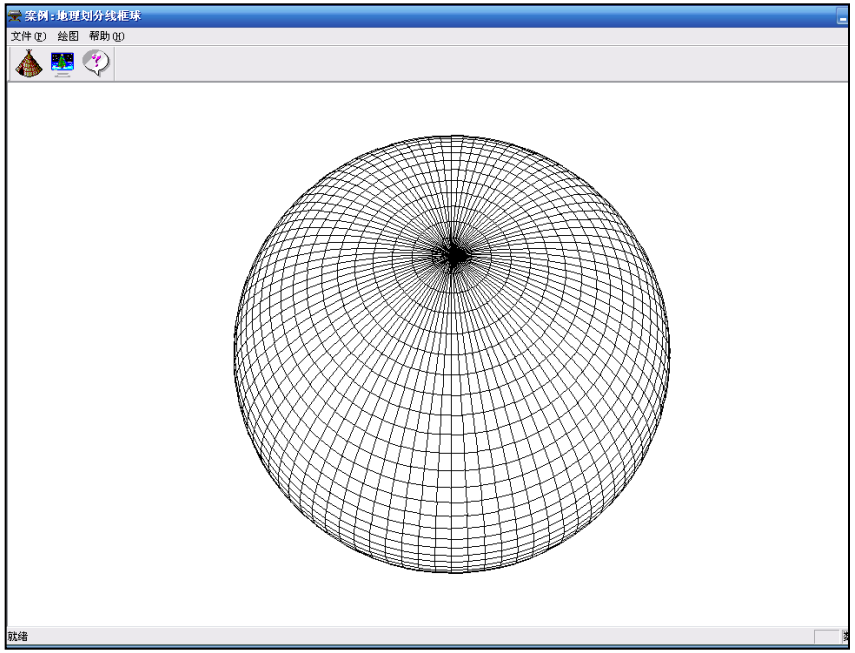


图9-31 南半球表面编号

球面网格化的方法分为： 地理划分法如图9-32所示。

递归划分法原理如图9-33所示，效果如图9-34所示。



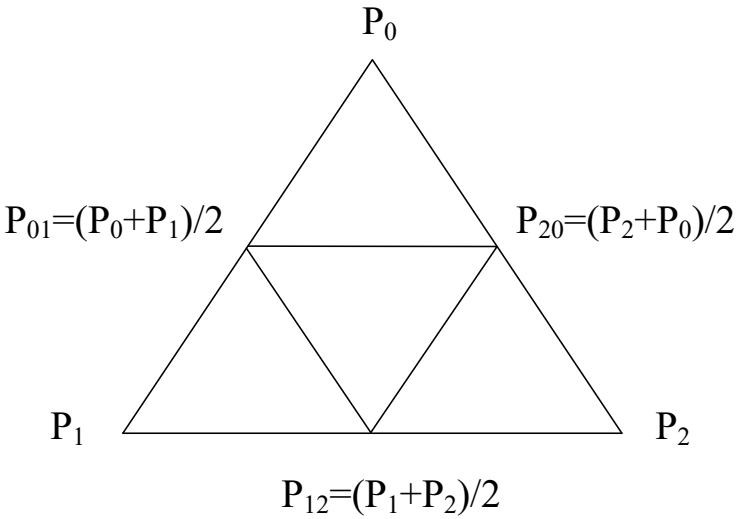
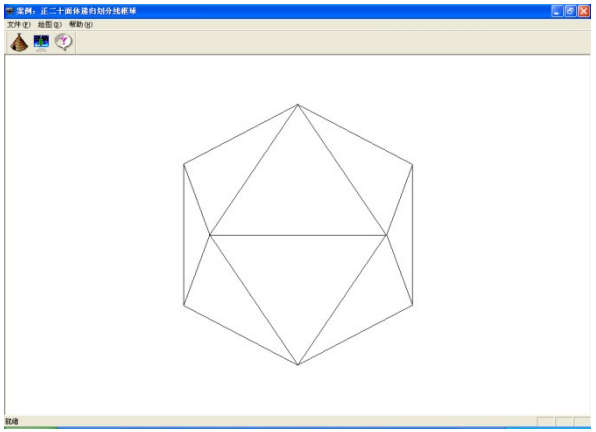
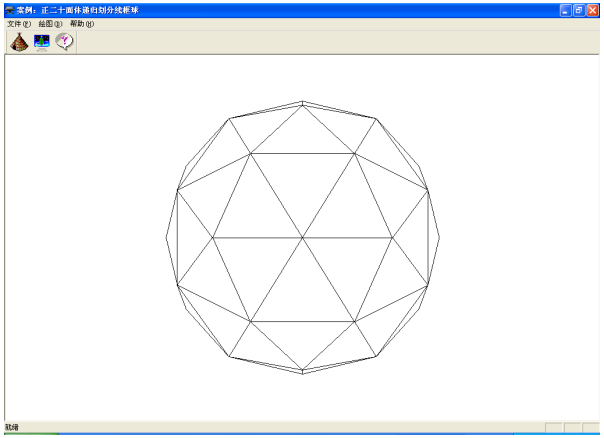


图9-32 地理划分法效果图

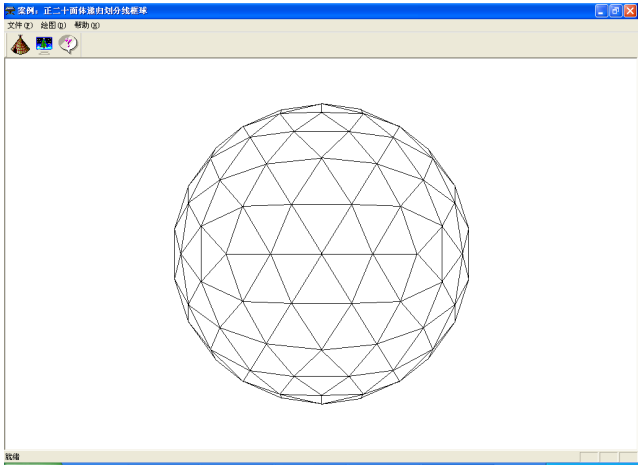
图9-33递归划分法效果图



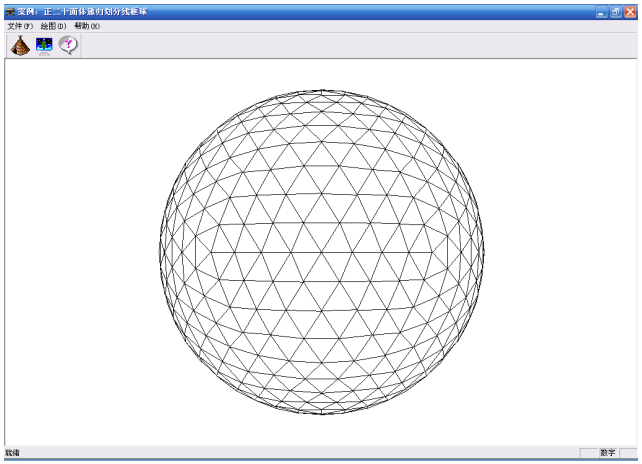
（a）递归深度n = 0



（b）递归深度n = 1



（c）递归深度n = 2



（d）递归深度n = 3

图9-34球面递归划分法效果图

〔2〕圆柱面

假定圆柱的中心轴与y轴重合，横截面是半径为r的圆，圆柱的高度沿着y轴方向从0拉伸到h，三维坐标系原点O位于底面中心，如图9-35所示。

如果不考虑顶面和底面，圆柱侧面的参数方程为

$$\begin{cases} x = r\cos\theta \\ z = r\sin\theta \end{cases} \qquad \begin{matrix} 0 \le \mathrm{y} \le h \\ 0 \le \theta \le 2\pi \end{matrix} \qquad (9\text{-}3)$$

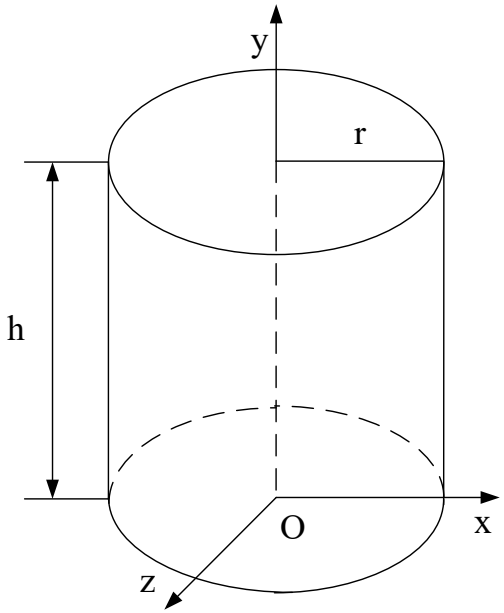


图9-35 圆柱面的数学模型

圆柱的离散化表示如图9-36所示 。假定圆柱的周向网格数$n_1 = 8$，纵向网格数$n_2 = 3$，圆柱网格模型的顶点表的顶点数为：$n_1 \times (n_2 + 1) + 2 = 34$。 顶面和底面各有8个三角形网格，侧面有24个四边形网格，圆柱网格模型的面表的面片总数为40。

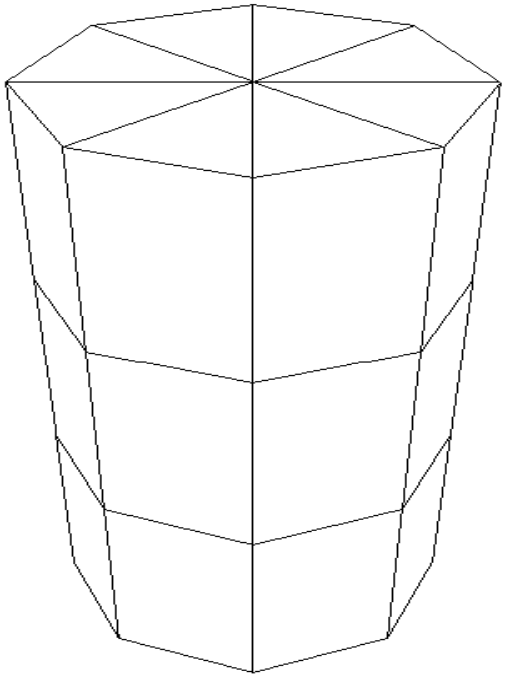


图9-36 示例圆柱

示例圆柱面的顶点和表面编号如图9-37～图9-39所示。

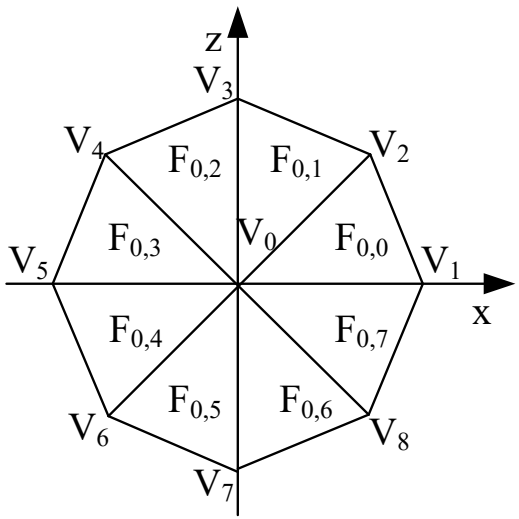


图9-37 圆柱底面的网格划分

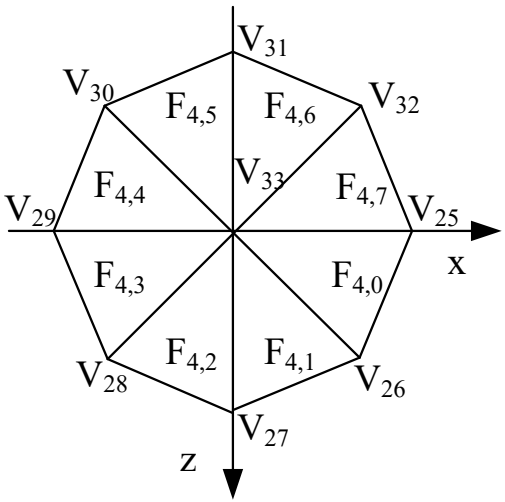


图9-38 圆柱的顶面的网格划分

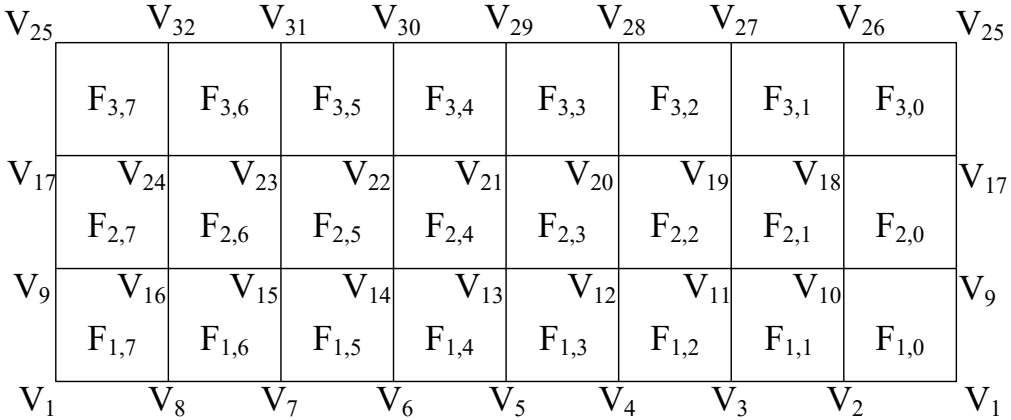


图9-39 圆柱侧面的网格划分

适当加大周向和纵向划分的网格数，圆柱面网格模型趋向光滑，透视投影效果如图9-39所示。

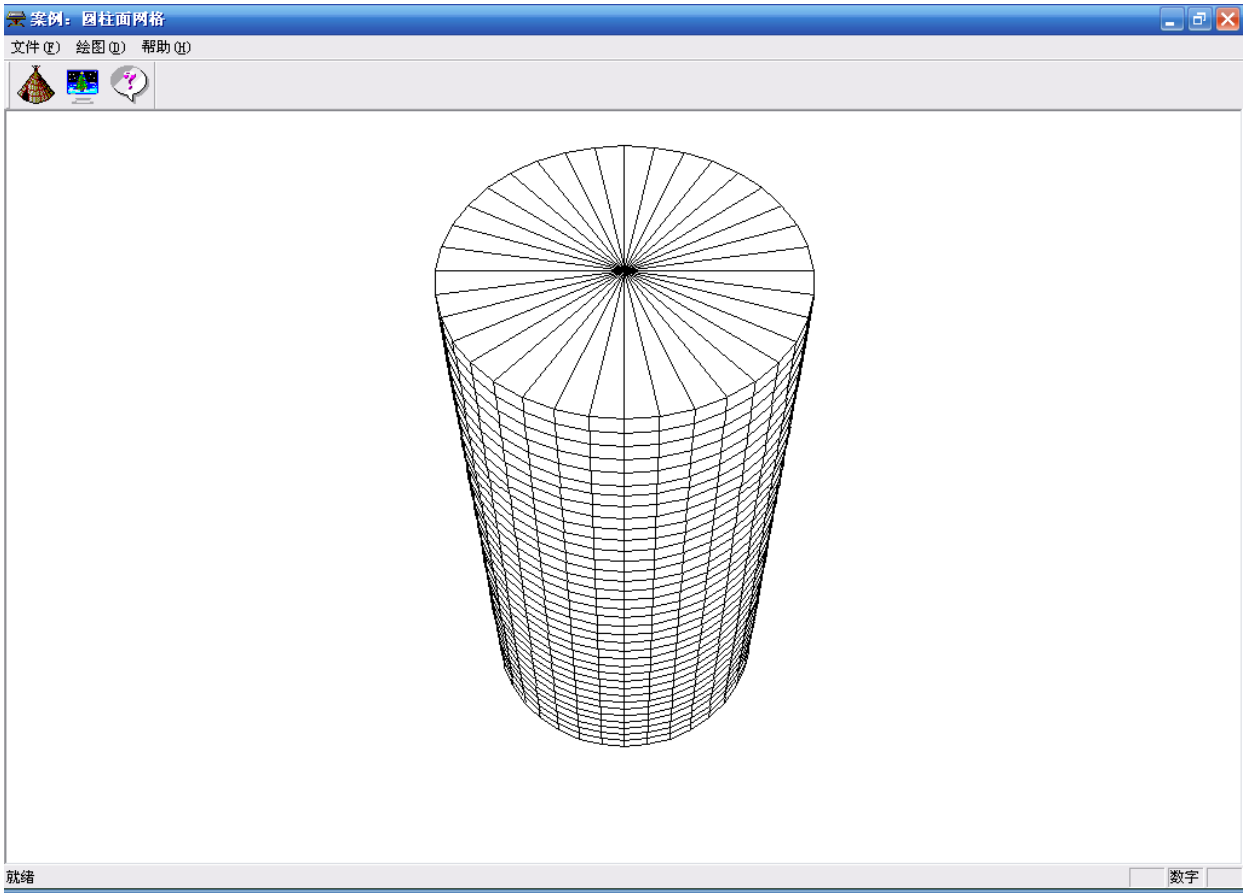


图9-40 圆柱的网格模型透视效果图

〔3〕圆锥面

假定圆锥的中心轴与y轴重合，横截面的最大半径为r，最小半径为0，圆锥的高度沿着y轴方向从0拉伸到h，三维坐标系原点O位于底面中心，如图9-41所示。

圆锥侧面的参数方程为

$$\begin{cases} x = (1-\dfrac{y}{h})r\cos\theta \\ z = (1-\dfrac{y}{h})r\sin\theta \end{cases} \qquad (9\text{-}4)$$

$$0 \le y \le h \qquad 0 \le \theta \le 2\pi$$

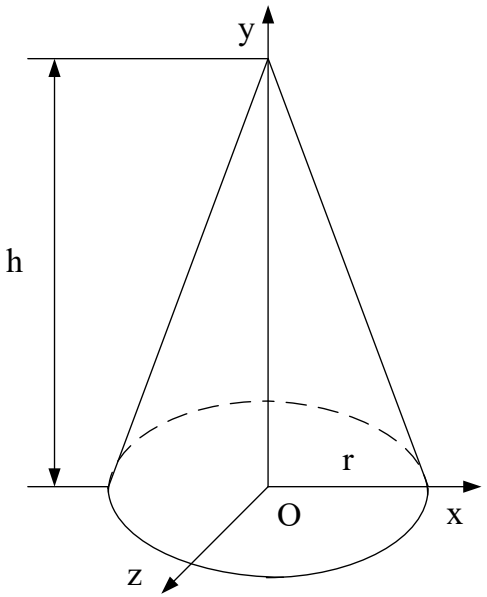


图9-41 圆锥面的数学模型

圆锥侧面的离散化表示如图9-42所示，侧面使用三角形网格和四边形网格逼近 。

圆锥底面使用三角形网格逼近，如图9-43所示。

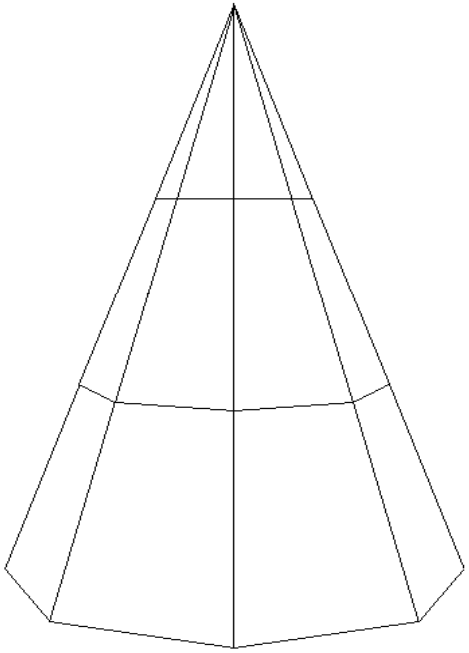


图9-42 示例圆锥

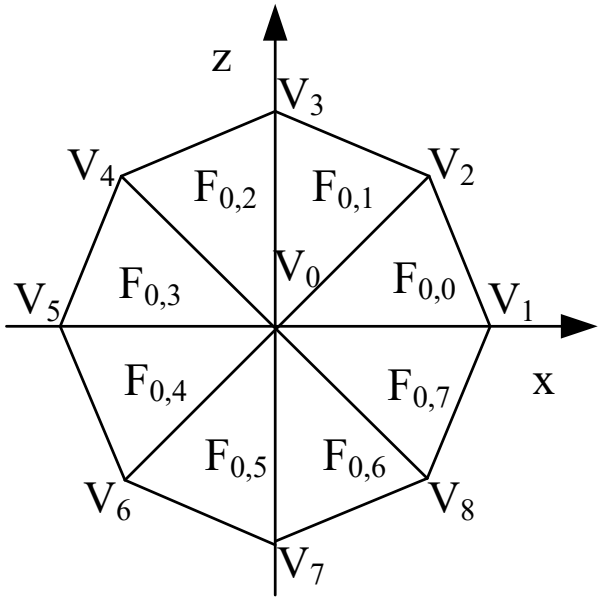


图9-43 圆锥底面的网格划分

假定圆锥的周向网格数$n_1 = 8$，纵向网格数$n_2 = 3$，网格侧面的顶点总数为： $n_1 \times n_2$，加上锥顶和底面的中心顶点，圆锥网格模型的顶点数为：$n_1 \times n_2 + 2 = 26$。圆锥底面划分为8个三角形网格，侧面有8个三角形网格和16个四边形网格，圆锥网格模型的面片总数为32。示例圆锥面的顶点和表面编号如图9-43～图9-45所示。

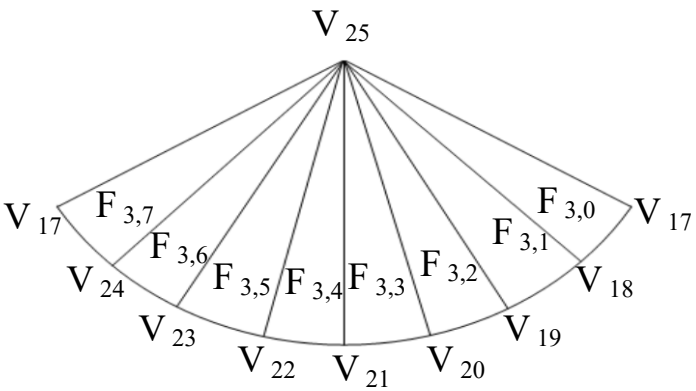


图9-44 圆锥侧面的三角形网格划分

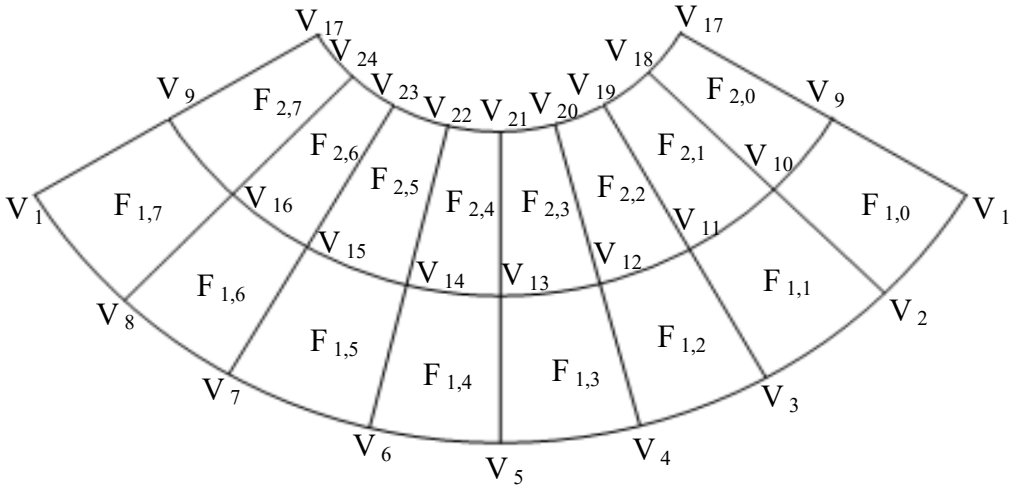
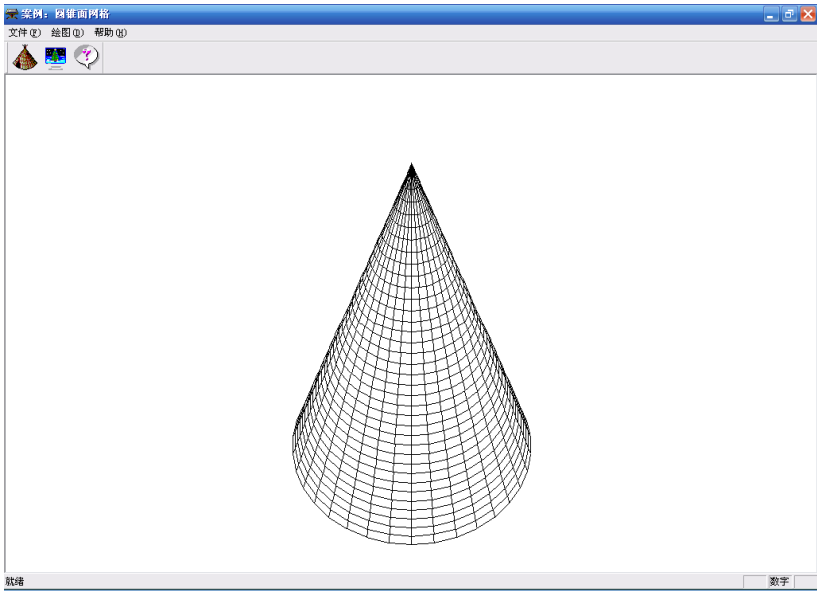


图9-45 圆锥侧面的四边形网格划分



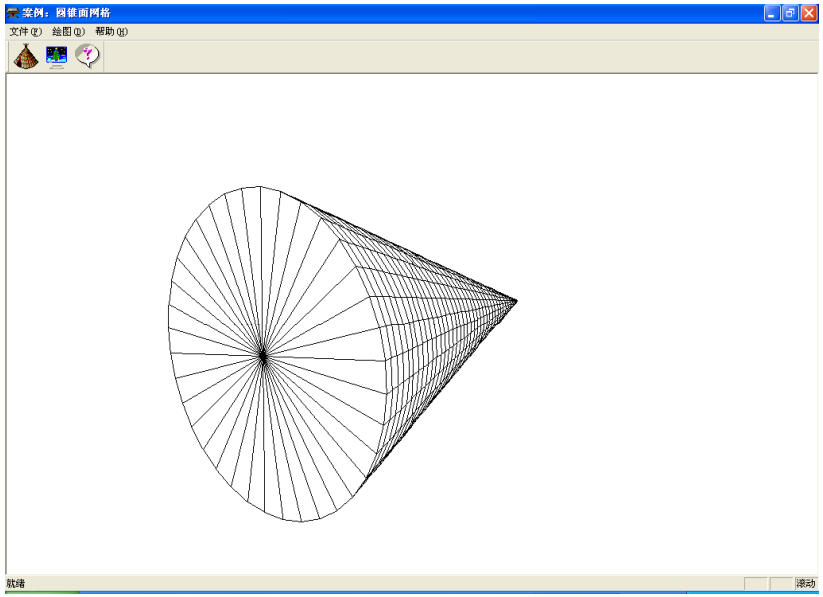


图9-46 圆锥的网格模型透视效果图

〔4〕圆环面

圆环面由一个在xOy面内偏置的圆周绕y轴进行旋转扫掠而成，如图9-47所示。环体半径为r_1，截面半径为r_2。

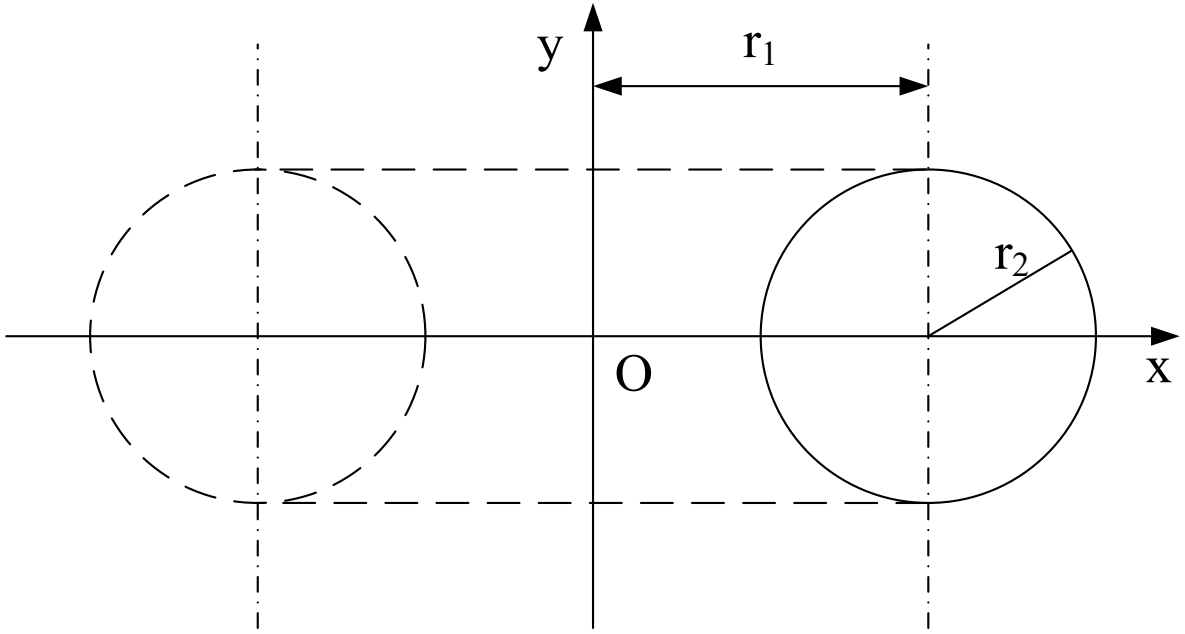


图9-47 实心偏置圆周扫掠成圆环面

建立右手坐标系Oxyz，原点位于圆环中心，x轴水平向右，y轴铅直向上，z轴指向观察者，圆环面的回转中心位于三维坐标系原点O。沿着环体的中心线建立右手动态参考坐标系O’x’y’z’，O’点位于环体的中心线上，x’轴沿着矢径O’O的方向向外，y’轴与y轴同向，z’轴沿着环体中心线的切线的顺时针方向，如图9-48所示。

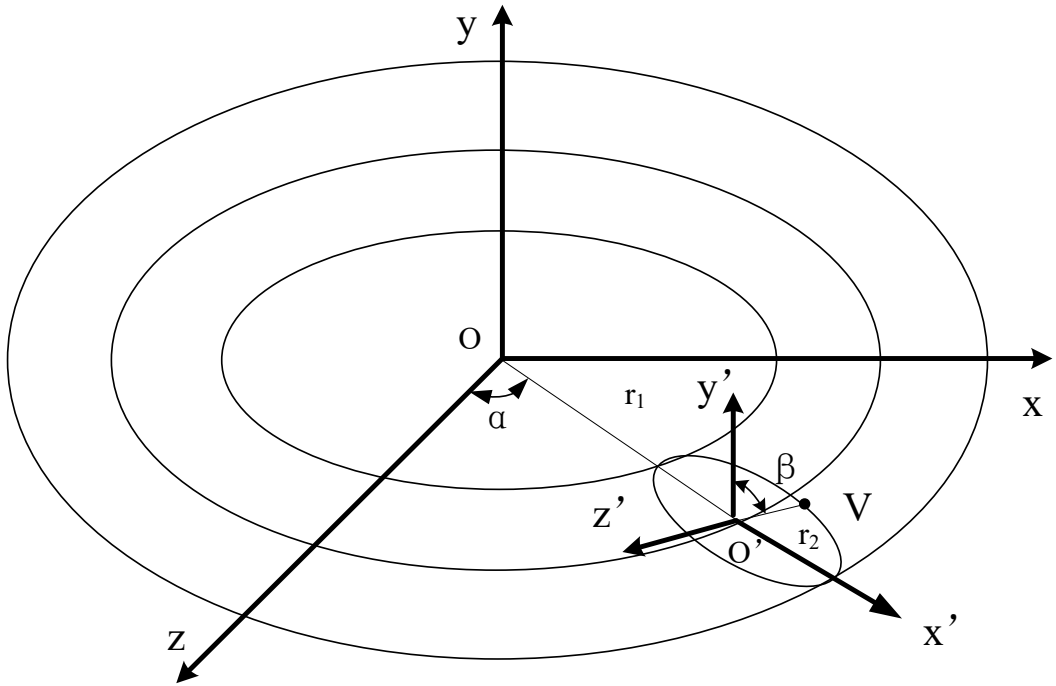


图9-48 圆环面的数学模型

圆环面的参数方程为

$$\begin{cases} x = (r_1 + r_2 \sin\beta)\sin\alpha \\ y = r_2 \cos\beta \\ z = (r_1 + r_2 \sin\beta)\cos\alpha \end{cases} \qquad 0 \le \alpha \le 2\pi, 0 \le \beta \le 2\pi \qquad （9\text{-}5）$$

示例圆环面的顶点编号和面表编号如图9-50所示。

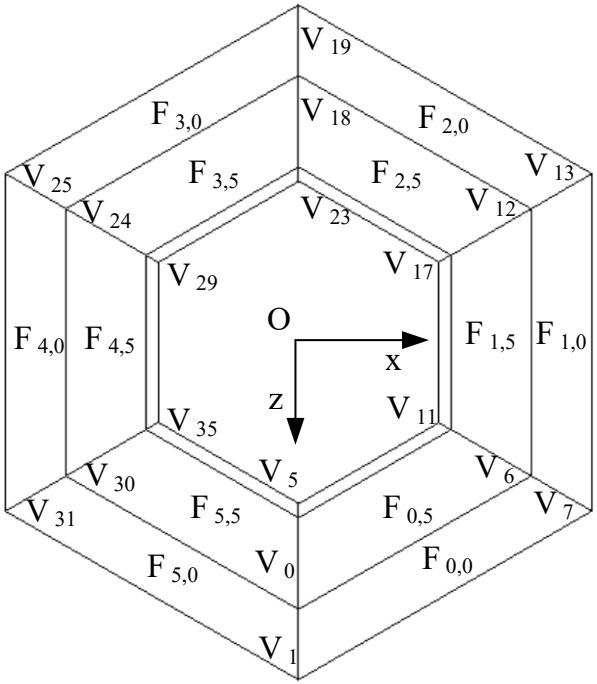
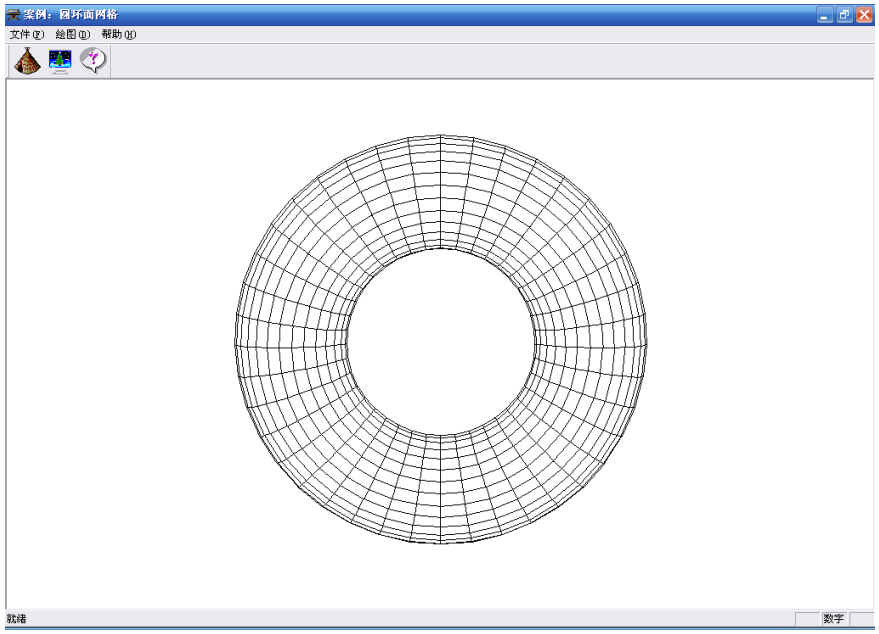


图9-50 圆环面的顶点和表面划分

适当加大周向和纵向划分的网格数，圆环面网格模型趋向光滑，透视投影效果如图9-51所示。



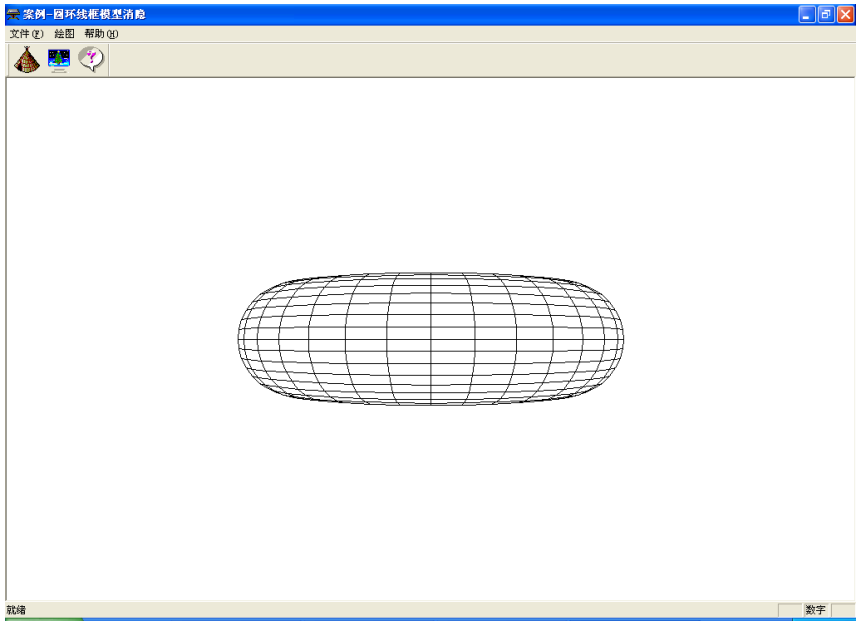


图9-50 圆环面的网格模型透视图

图9-50 消隐环

9.2 消隐算法分类

根据消隐方法的不同，消隐算法可分为两类：

（1）**隐线算法**。用于消除物体上不可见的边界线。隐线算法主要是针对线框模型提出的，它只要求画出物体的各可见棱边，如图9-51所示。

（2）**隐面算法**。用于消除物体上不可见的表面。隐面算法主要是针对表面模型提出的，一般不绘制物体的可见棱边，只使用指定颜色填充物体的各可见表面，如图9-52所示。

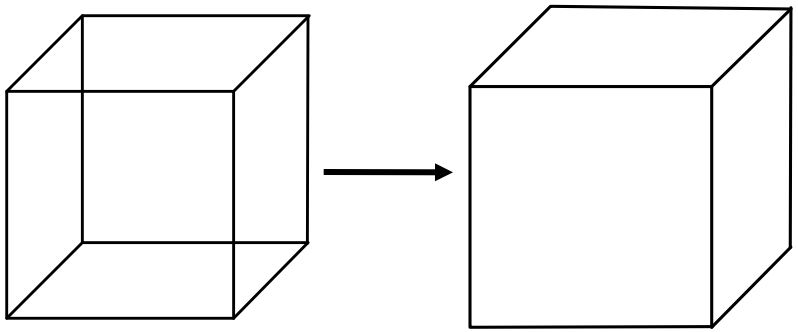


图9-51 隐线算法

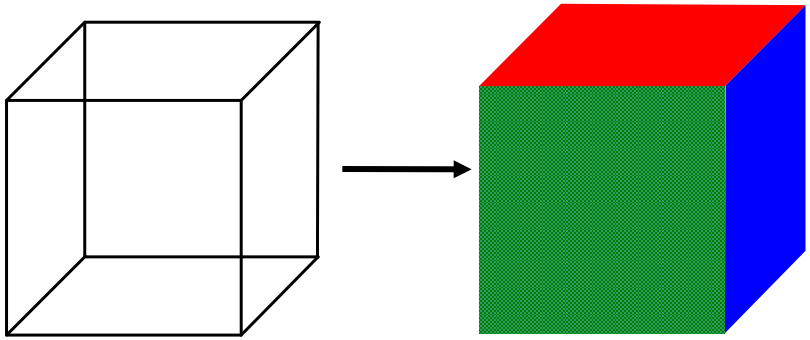


图9-52 隐面算法

计算机图形学的创始人Sutherland根据消隐空间的不同，将消隐算法分为3类：

（1）物体空间法。物体空间消隐算法主要在三维观察空间中完成。根据模型的几何关系来判断哪些表面可见，哪些表面不可见。

（2）图像空间法。图像空间消隐算法主要在物体投影后的二维图像空间中利用帧缓冲信息确定哪些表面遮挡了其它表面。图像空间法受限于显示器的分辨率。

（3）物像空间法。在描述物体的三维观察空间和二维图像空间中同时进行消隐。

9.3 隐线算法

9.3.1凸多面体消隐算法

对于凸多面体的任一个面，根据其外法矢量和视矢量的夹角θ来进行可见性检测。如果两个矢量的夹角0°≤θ≤90°时，表示该表面可见；如果90° < θ≤180°时，表示该表面不可见。

以图9-53所示的立方体为例来进行具体说明。

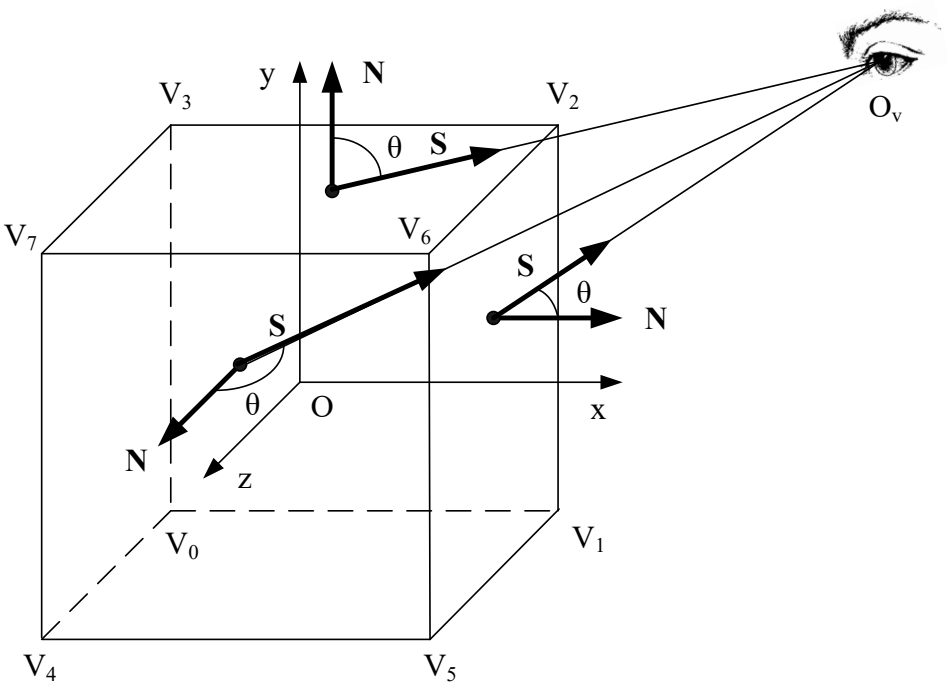


图9-53 凸多面体消隐原理

通过各个表面的三维顶点坐标，可以计算该表面的外法矢量。对于“前面”$V_4V_5V_6V_7$，取V_4点为参考点。

该点的外法矢量 $N = \vec{V_4V_5} \times \vec{V_4V_6}$

$$\vec{V_4V_5} = (x_5 - x_4, y_5 - y_4, z_5 - z_4) \qquad \vec{V_4V_6} = (x_6 - x_4, y_6 - y_4, z_6 - z_4)$$

$$N_x = (y_5 - y_4)(z_6 - z_4) - (z_5 - z_4)(y_6 - y_4)$$

$$N_y = (z_5 - z_4)(x_6 - x_4) - (x_5 - x_4)(z_6 - z_4)$$

$$N_z = (x_5 - x_4)(y_6 - y_4) - (y_5 - y_4)(x_6 - x_4)$$

前面的外法矢量可以表示为：$N = N_x i + N_y j + N_z k$ （9-6）

式中， i，j，k为三维坐标系的标准单位矢量

给定视点位置球面坐标表示为：

（ $R\sin\varphi\sin\theta$ ， $R\cos\varphi$ ，$R\sin\varphi\cos\theta$ ）

其中：R为视径，$0 \le \varphi \le \pi$ ， $0 \le \theta \le 2\pi$ 。

视矢量从多边形的参考点V_4的指向视点，视矢量分量的计算公式为：

$$S = (R \cdot \sin\varphi\sin\theta - x_4, R\cos\varphi - y_4, R\sin\varphi\cos\theta - z_4)$$

视矢量表示为：

$$S = S_x i + S_y j + S_z k \qquad (9\text{-}7)$$

式中， i，j，k为三维坐标系的标准单位矢量。

表面外法矢量和视矢量的数量积为：

$$N \cdot S = N_x \cdot S_x + N_y \cdot S_y + N_z \cdot S_z$$

将外法矢量N和视矢量S单位化为n和s，则有

$$n \cdot s = \cos\theta = n_x \cdot s_x + n_y \cdot s_y + n_z \cdot s_z \qquad （9\text{-}8）$$

可见cosθ的正负取决于表面的单位外法矢量和单位视矢量的数量积： $n_x \cdot s_x + n_y \cdot s_y + n_z \cdot s_z$

凸多面体表面可见性检测条件如下：

当0°≤θ < 90°时， $n_x \cdot s_x + n_y \cdot s_y + n_z \cdot s_z > 0$

表面可见，画出表面多边形边界。

当θ=90°时， $n_x \cdot s_x + n_y \cdot s_y + n_z \cdot s_z = 0$

表面外法矢量与视矢量垂直，表面多边形退化为一条直线，绘制结果为一段直线；

当90° < θ≤180°时， $n_x \cdot s_x + n_y \cdot s_y + n_z \cdot s_z < 0$

凸多面体表面不可见，不绘制该多边形边界。

因此，可以将 $n \cdot s \geq 0$ 作为绘制表面的基本条件。

对于立方体而言，使用 $n \cdot s \geq 0$ 剔除了背向视点的不可见表面

只绘制朝向视点的3个可见表面。因此本算法也被称为背面剔除（back culling）算法。

立方体消隐前的透视变换图如图9-54所示，画出了全部6个表面的线框；消隐后的透视变换图如图9-55所示，只画出可见的表面棱边。

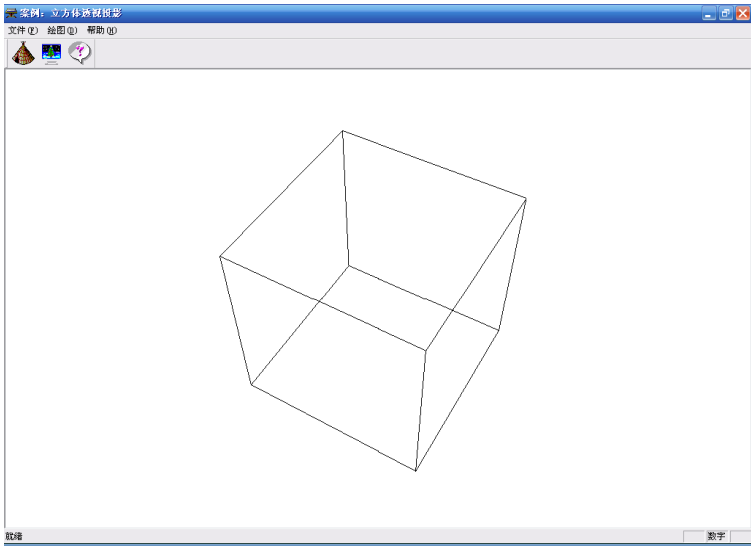


图9-54 消隐前的立方体透视投影

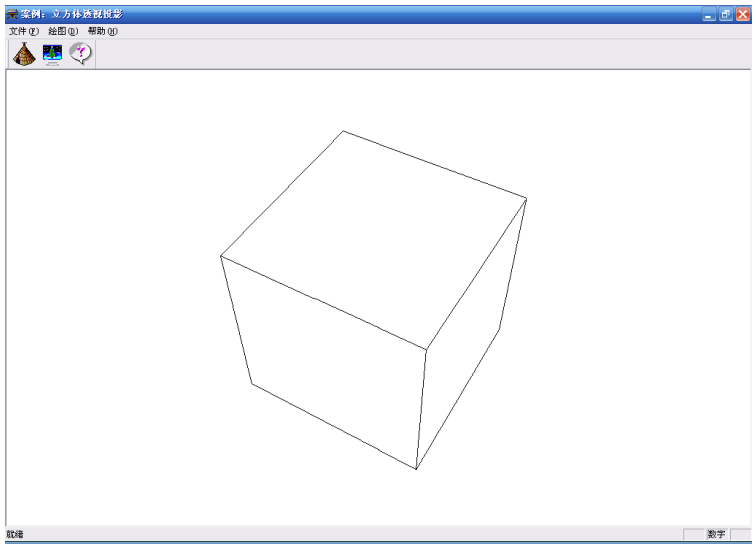


图9-55 消隐后的立方体透视投影

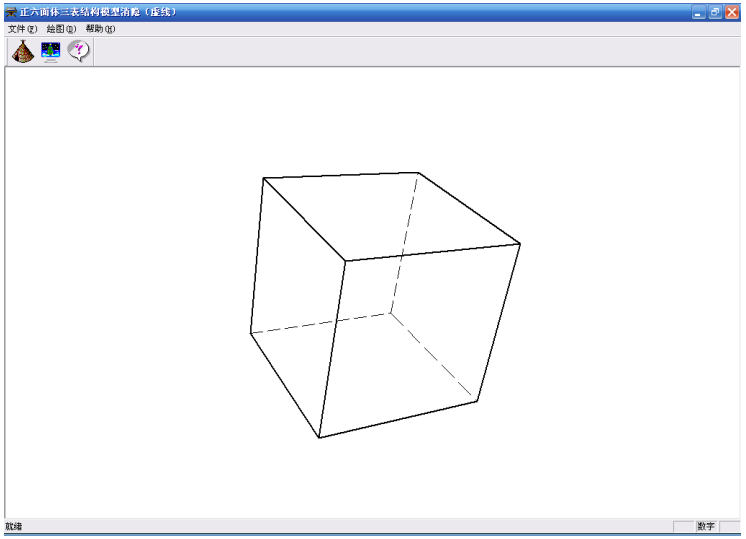


图9-57 立方体虚线消隐

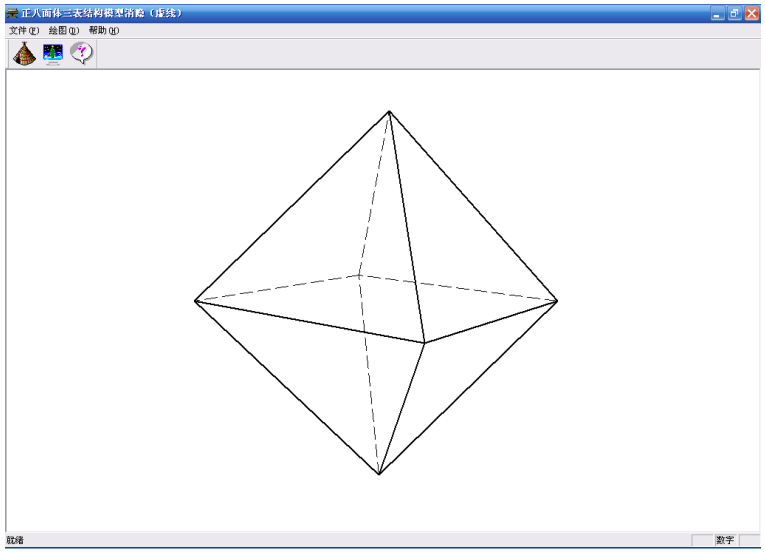


图9-58 正八面体虚线消隐

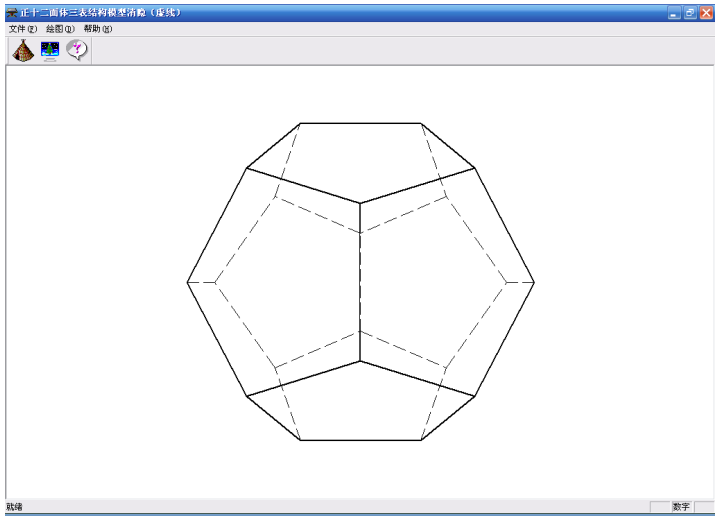


图9-59 正十二面体虚线消隐

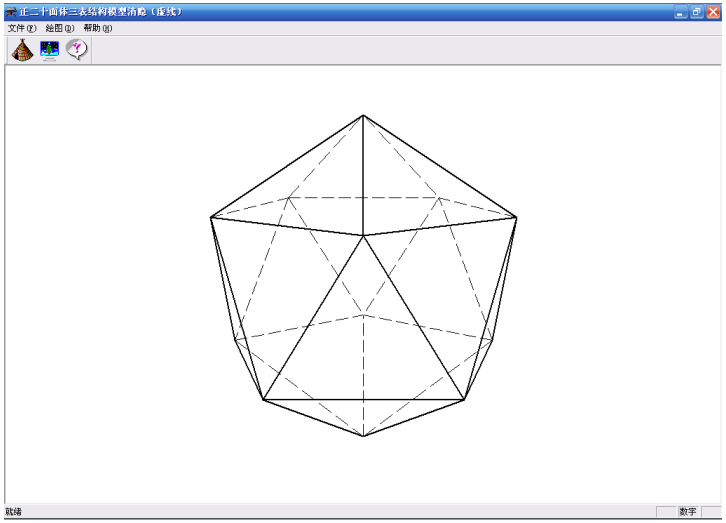


图9-60 正二十面体虚线消隐

9.3.2 曲面体消隐算法

曲面体可以采用有限单元法划分为若干个小平面区域。常采用四边形平面片或三角形平面片来逼近曲面体。

消隐主要是确定各四边形平面片或三角形平面片的可见性，与凸多面体消隐类似，即用外法矢量和视矢量的数量积来进行可见性检测。

球面可用α参数簇和β参数曲线簇所构成的四边形经纬网格来表示，如图9-61所示。设相邻的两条纬线分别为α_0、 α_1，相邻的两条经线分别为β_0、 β_1，则四边形平面片$V_0V_1V_2V_3$各点的坐标为：$V_0(\alpha_0,\beta_0)$、$V_1(\alpha_1,\beta_0)$、 $V_2(\alpha_1,\beta_1)$、$V_3(\alpha_0, \beta_1)$。

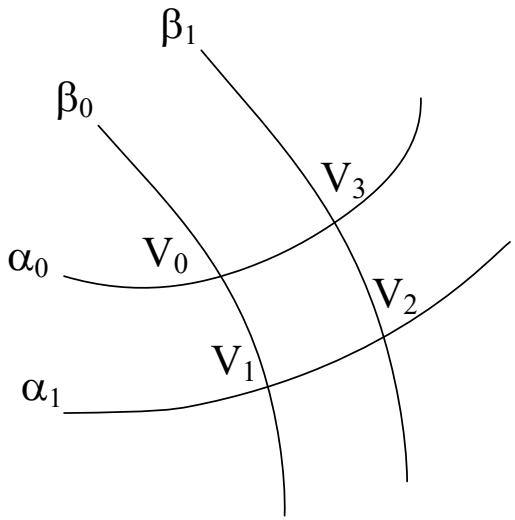


图9-60 球的经纬网格

以V_1V_0和V_2V_0为边矢量，计算四边形平面片的$V_0V_1V_2V_3$外法矢量为：

$$\vec{N} = \vec{V_0V_1} \times \vec{V_0V_2} \qquad (9\text{-}9)$$

给定视点位置球面坐标表示为：（ $R\sin\varphi\sin\theta$ ，$R\sin\varphi\sin\theta$ ， $R\sin\varphi\cos\theta$ ）

对于四边形平面片$V_0V_1V_2V_3$的参考点V_0（α_0，β_0），视矢量分量的计算公式为

$$S_x = R\sin\varphi\sin\theta - r\cdot\sin\alpha_0\cdot\sin\beta_0$$

$$S_y = R\cos\varphi - r\cos\alpha_0$$

$$S_z = R\sin\varphi\cos\theta - r\sin\alpha_0\cos\beta_0$$

式中，R为视点的矢径，φ和θ为视点的位置角。r为球面的半径，α_0和β_0为球面上一点V_0的位置角。

四边形平面片$V_0V_1V_2V_3$的参考点V_0（α_0，β_0）的法矢量N的计算方法与凸多面体类似。球面上V_0（α_0，β_0）点的平均外法矢量可以使用该点的位置矢量代替。

将法矢量N规范化为单位矢量n，视矢量S规范化为单位矢量s，有

$$n\cdot s = n_x\cdot s_x + n_y\cdot s_y + n_z\cdot s_z$$

球面网格四边形平面片可见性检测条件为：

当 绘制该面片。

球面线框模型消隐前的透视变换图如图9-62所示，北极点和南极点同时绘制出来，无法确认北极点面向读者还是南极点面向读者 。

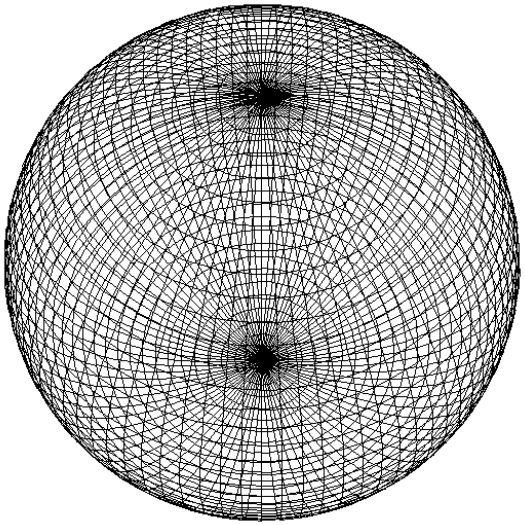


图9-62 消隐前的球面透视投影

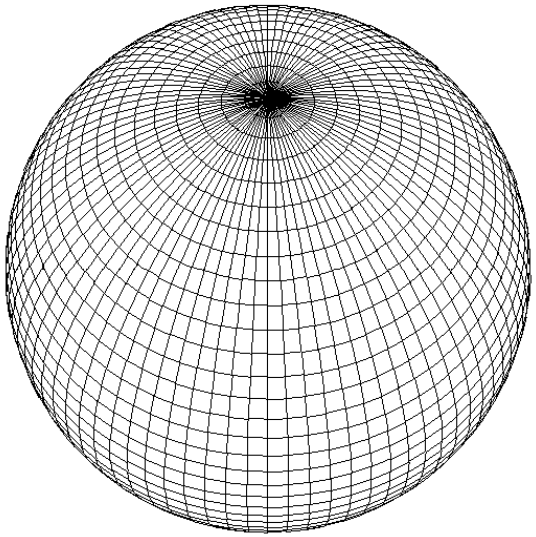


图9-63 消隐后的球面透视投影

使用背面剔除算法后，可以看出图9-62中球面的北极点面向读者，如图9-63所示。

9.4 隐面算法

从视点的角度观察物体的表面，离视点近的表面的投影遮挡了离视点远的表面的投影，屏幕上的绘制结果为所有可见表面最终投影的集合。

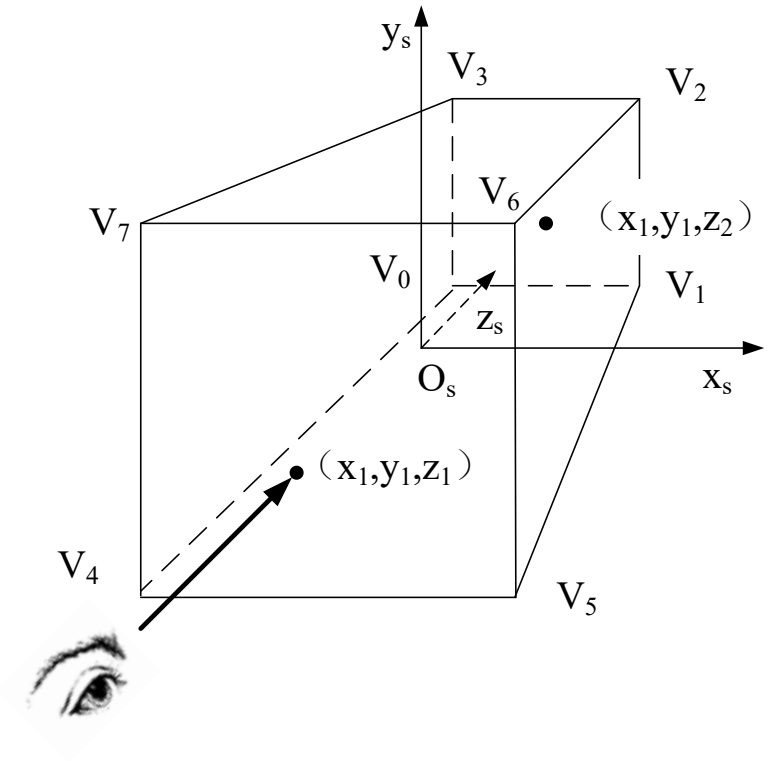
常用的消隐有两种：深度缓冲器算法和深度排序算法。

9.4.1 深度缓冲器算法

1.算法原理

深度缓冲器算法于1974年由Catmull提出，属于图像空间消隐算法。在物空间内不对物体表面的可见性进行检测，在像空间中根据每个像素的深度值确定最终绘制的物体表面上各个像素的颜色。也称为Z-Buffer算法。

Edwin Earl Catmull



Z-Buffer算法建立两个缓冲器：

深度缓冲器，用以存储图像空间中每一像素相应的深度值，初始化为最大深度值（z_s坐标）。

帧缓冲器，用以存储图像空间中的每个像素的颜色，初始化为屏幕的背景色。

2.算法描述

① 帧缓冲器初始值置为背景色。

② 确定深度缓冲器的宽度、高度和初始深度。一般将初始深度置为最大深度值。

③ 对于多边形表面中的每一像素（x_s,y_s），计算其深度值z_s（x_s,y_s）。

④ 将z_s（x_s,y_s）与存储在z缓冲器中该位置的深度值zBuffer（x_s,y_s）进行比较。

⑤ 如果z_s（x_s,y_s）≤zBuffer（x_s,y_s），则将此像素的颜色写入帧缓冲器，且用z（x_s,y_s）重置zbuffer（x_s,y_s）。

3.计算深度

若多边形的平面方程已知，一般采用增量法计算扫描线上每一像素点的深度值。当立方体旋转到图9-65所示的位置时，六个表面都不与投影面xOy面平行，这时需要根据每个表面的平面方程计算多边形内各个像素点的深度值。

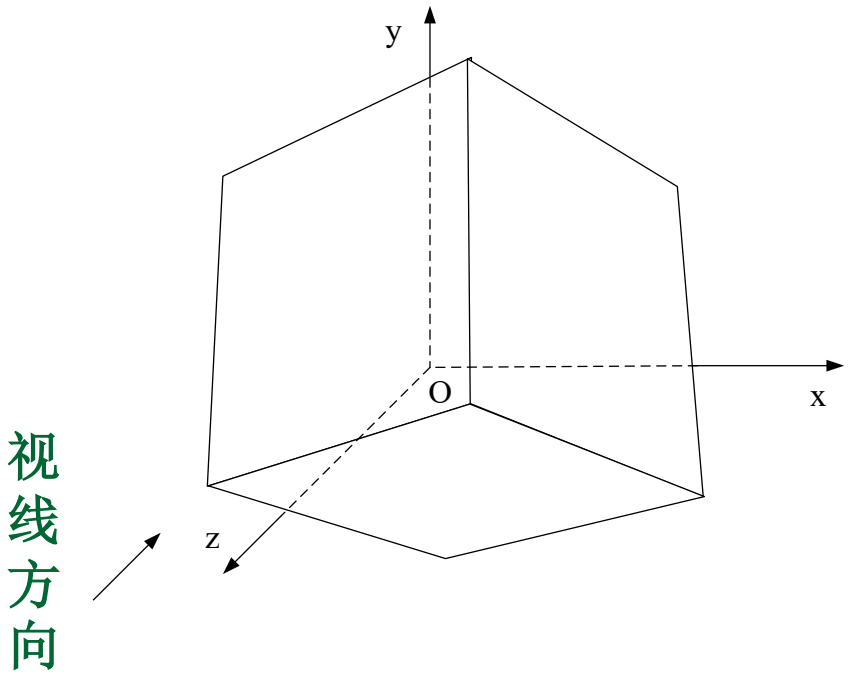


图9-65 旋转立方体

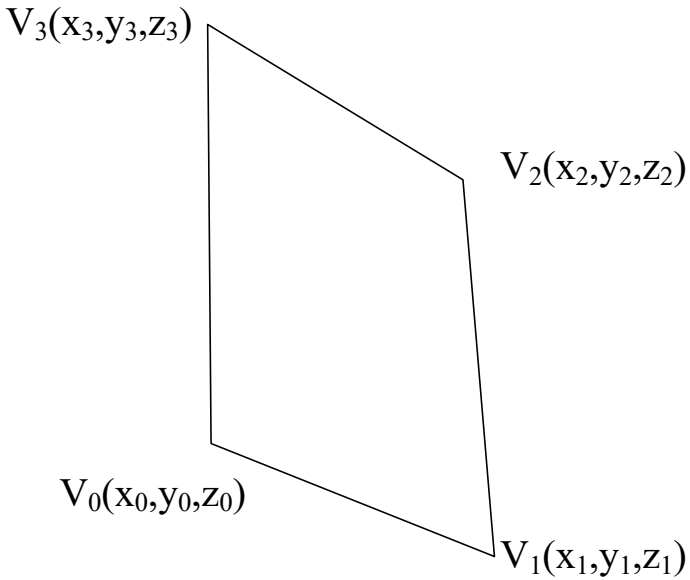


图9-66 旋转立方体的任一表面

① 图9-66所示的一个立方体表面，其平面一般方程为：

$$Ax + By + Cz + D = 0 \qquad (9\text{-}10)$$

系数A，B，C是该平面的一个法矢量N的坐标，即

$$N = \{A, B, C\}$$

② 根据表面顶点的坐标可以计算出两个边矢量

$$\overrightarrow{V_0V_1} = \{x_1 - x_0, y_1 - y_0, z_1 - z_0\}$$

$$\overrightarrow{V_0V_2} = \{x_2 - x_0, y_2 - y_0, z_2 - z_0\}$$

③ 根据两个边矢量的叉积，可求得表面的法矢量N，得到系数A，B，C

$$B=(z_1-z_0)\cdot(x_2-x_0)-(x_1-x_0)\cdot(z_2-z_0)$$

$$A=(y_1-y_0)\cdot(z_2-z_0)-(z_1-z_0)\cdot(y_1-y_0)$$

$$C=(x_1-x_0)\cdot(y_2-y_0)-(y_1-y_0)\cdot(x_2-x_0)$$

④ 将A、B、C和点（x_0,y_0,z_0）代入方程（9-10），得

$$D=-Ax_0-By_0-Cz_0$$

⑤ 从方程（9-10）得到当前像素点（x，y）处的深度值

$$z(x,y)=-\frac{(Ax+By+D)}{C}$$

⑥ 已知扫描线y_i与多边形相交的左边界的像素（x_i，y_i）的深度值为z（x_i，y_i），其相邻点（x_{i+1}，y_i）处的深度值为z（x_{i+1}，y_i）。由此可以计算出该扫描线上的所有后续像素点的深度值。同一扫描线上的深度增量可由一步加法完成。

$$z(x_i+1, y_i) = \frac{-A(x_i+1) - By_i - D}{C} = z(x_i, y_i) - \frac{A}{C}$$

式中，$-\frac{A}{C}$为深度步长。

⑦ 下一条扫描线$y = y_{i+1}$，其最左边的像素点坐标的x值为

$$x(y_{i+1}) = x(y_i) + \frac{1}{k}$$

式中,k为有效边的斜率

- 用深度d初始化宽度为w、高度为h的深度缓冲器的代码如下。

```
void CZBuffer::InitDeepBuffer(int w,int h,double d)
                                                    //初始化深度缓冲器
{
        zBuffer=new double *[ w];
         for(int i=0;i<w;i++)
                zBuffer[i]=new double[h];
         for(i=0;i< w;i++)
                for(int j=0;j< h;j++)
                        zBuffer[i][j]=double(d);
}
```

- 一般将物体的回转中心放置在自定义的屏幕坐标系原点，也即位于屏幕中心。为了避免深度缓冲区数组zBuffer下标的索引号为负值，二维深度缓冲区数组采用zBuffer[xs+w/2][ys+h/2]进行匹配。

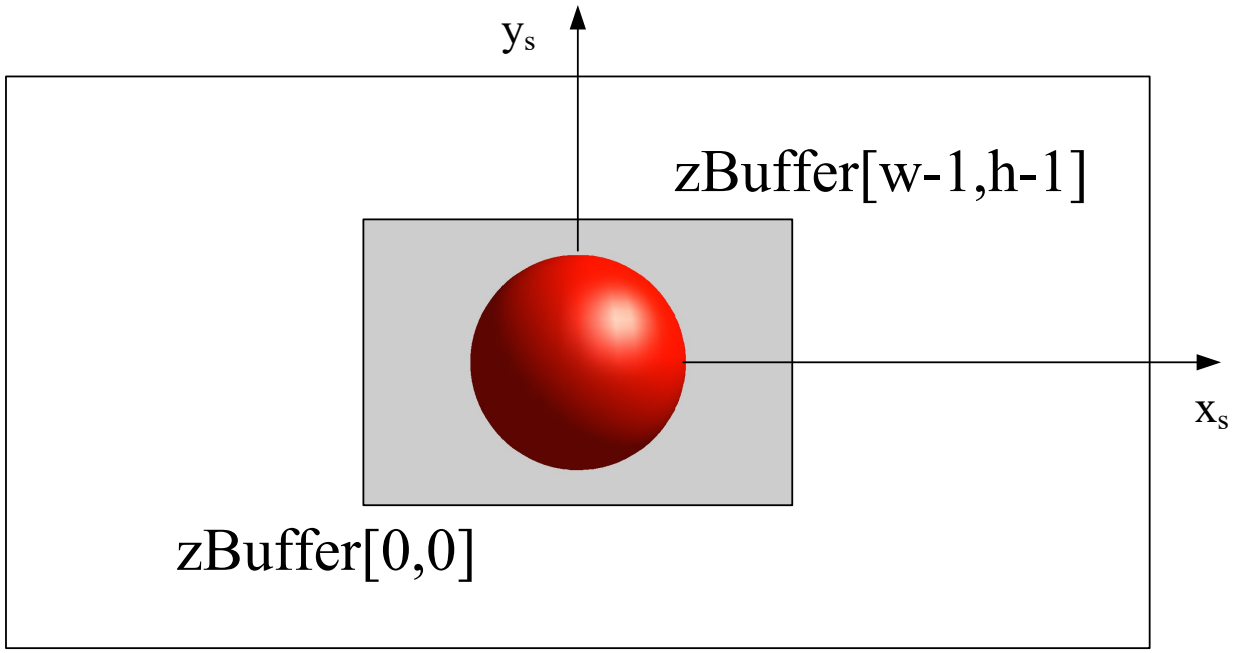


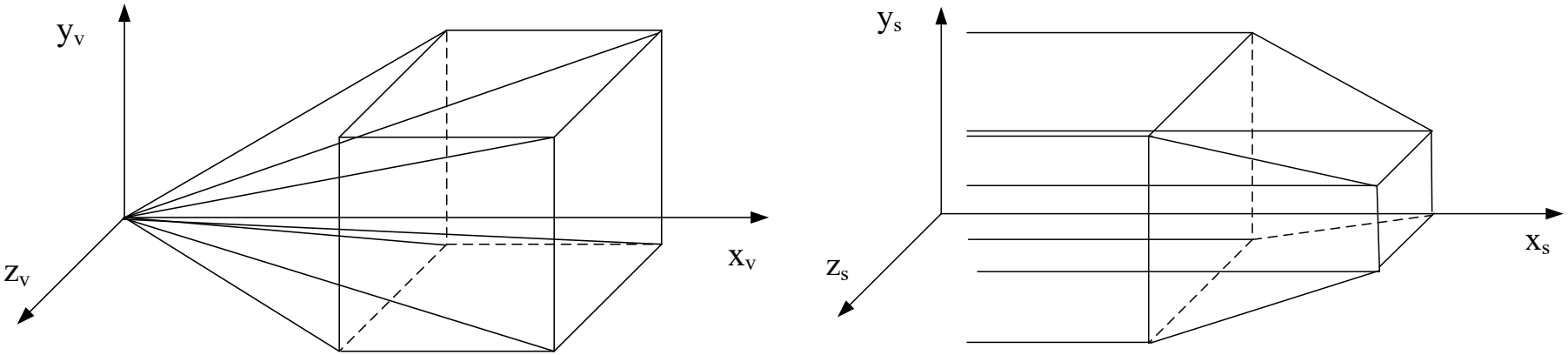
图9-67 匹配深度缓冲器数组的下标

- 真实感场景中一般绘制的是物体的透视投影，在第6章中公式（6-55）仅计算了屏幕坐标系的二维坐标。使用深度缓冲器算法绘制物体的透视图时，需要在透视变换后保留物体的深度值。设视域四棱台的近剪切面为Near，远剪切面为Far

$$\begin{cases} x_s = Near \cdot \dfrac{x_v}{z_v} \\ y_s = Near \cdot \dfrac{y_v}{z_v} \\ z_s = Far \cdot \dfrac{1 - Near/z_v}{Far - Near} \end{cases}$$

式中（x_v,y_v,z_v）为观察坐标系中的坐标，（x_s,y_s,z_s）为屏幕坐标系中的坐标。

- 图9-68中，观察坐标系中从视点出发的视线会聚于视点。屏幕坐标系中，视线被映射为平行线。因而，只有在屏幕坐标系中具有相同（x_s,y_s）的点才可能发生遮挡，判别一个点是否位于另一个点的前面则可简化为z_s值的比较。



（a）观察坐标系　　（b）屏幕坐标系

图9-68 立方体视线的变换

- 图9-69中，红、绿、蓝三角形的深度相互交叉，无法区分前后顺序。使用z-buffer算法根据三角形上每一点的深度值填充颜色，则可以很方便地消隐 。

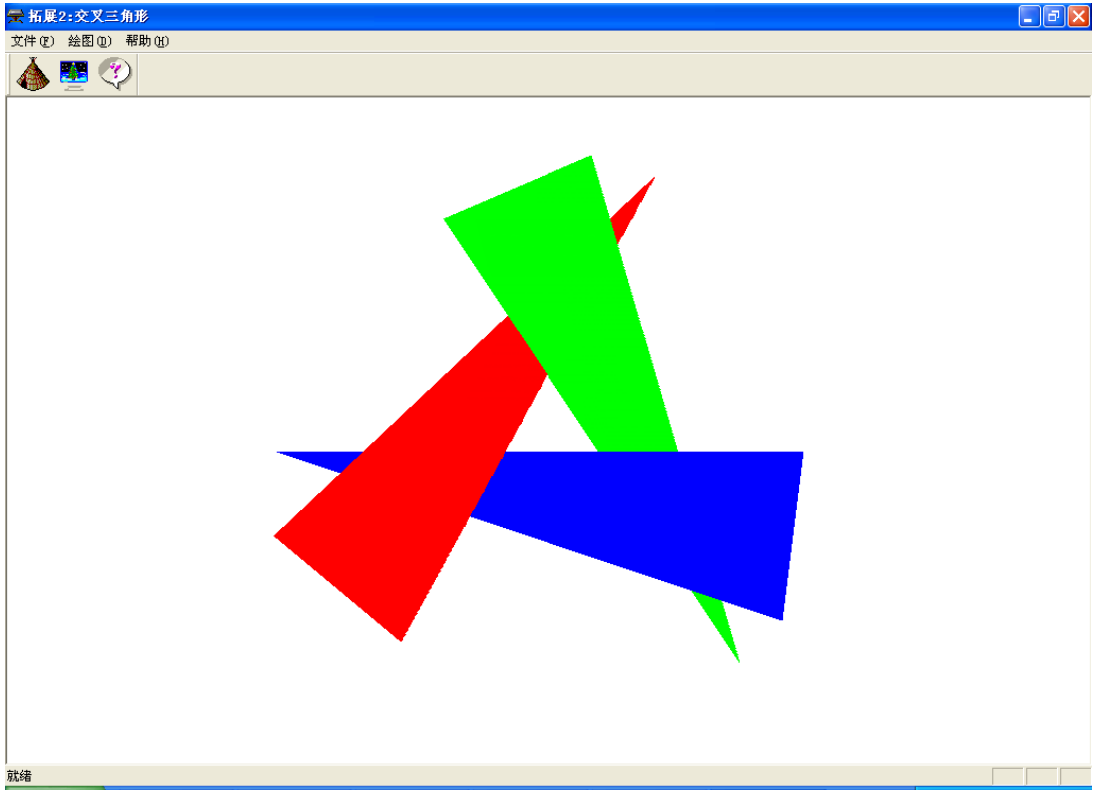


图9-69 交叉三角形消隐

一些特殊的凹多面体如圆环，绘制线框模型时，使用背面剔除算法并不能完全消除隐藏线。当环面垂直于于投影面时，消隐结果存在错误，如图9-70所示。原因是保留了内环面的前面和外环面的前面。圆环线框模型的消隐需要使用专门针对凹多边形设计的算法实现。但如使用z-buffer算法绘制圆环的表面模型，则可以得到正确结果，如图9-71所示，

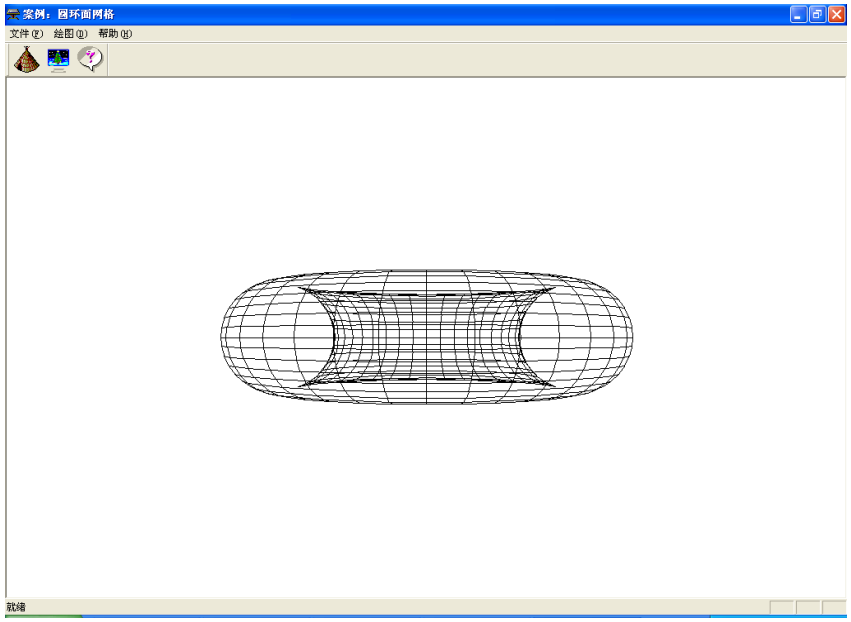
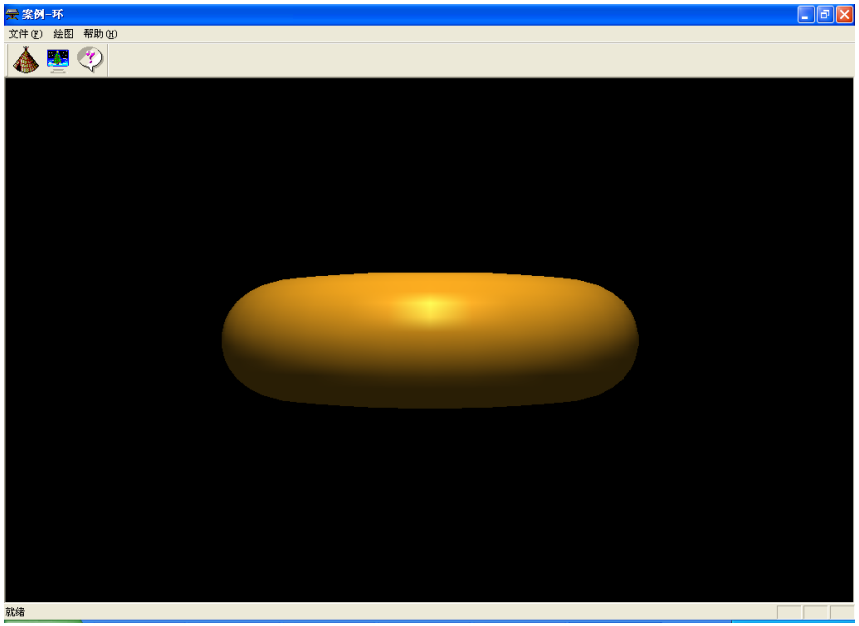


图9-70 圆环线框模型消隐



消隐 图9-71 圆环表面模型消隐

9.4.2 深度排序算法

深度排序算法是同时运用物体空间和图像空间的消隐算法。在物体空间中将表面按深度优先级排序，然后在图像空间中，由深度最大的表面开始，依次绘制各个表面。这种消隐算法通常被称为画家算法。

深度排序算法的原理是：先把屏幕置成背景色，再把物体的各个面按其离视点的远近排序形成深度优先级表，离视点远者位于表头，离视点近者位于表尾。然后按照从表头到表尾的顺序绘制各个表面，后画的表面颜色取代先画的表面颜色，相当于消除了隐藏面。

在算法上需要构造顶点表、面表来实现。

深度优先级排序算法的难点在于确定物体的深度优先级。

①对于图9-72所示的4个条相互叠压（称为叠压条），每个条有一个独立的深度，而且4个条的深度彼此不同，则可以直接建立一个确定的深度优先级表。

②对于图9-73的4个条相互交叉（称为交叉条），每个条至少有两个深度，不能简单地建立深度优先级表。

解决方法是沿着图中虚线循环地分割每个条，直至最终可建立确定的深度优先级表，另一种解决方法是使用深度缓冲器算法直接绘制交叉条。使用深度排序算法绘制的叠压条如图9-74所示。使用深度缓冲算法绘制的交叉条如图9-75所示。

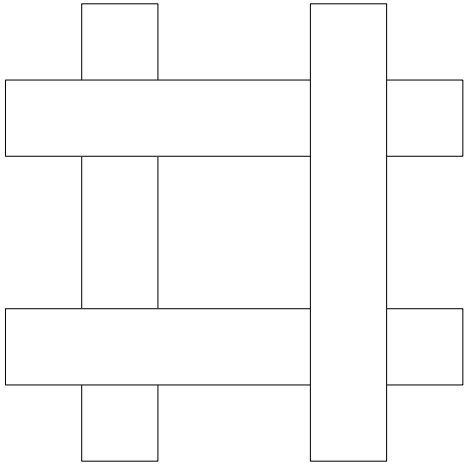


图9-72 叠压条线框模型

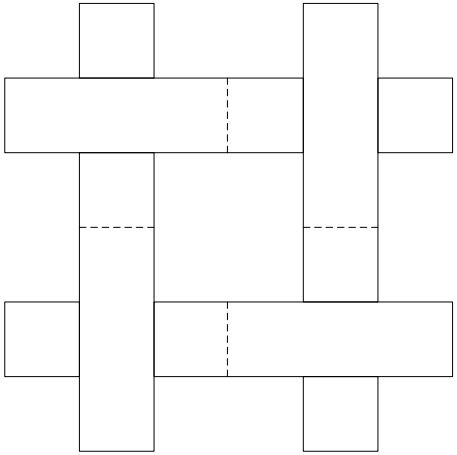


图9-73 交叉条线框模型

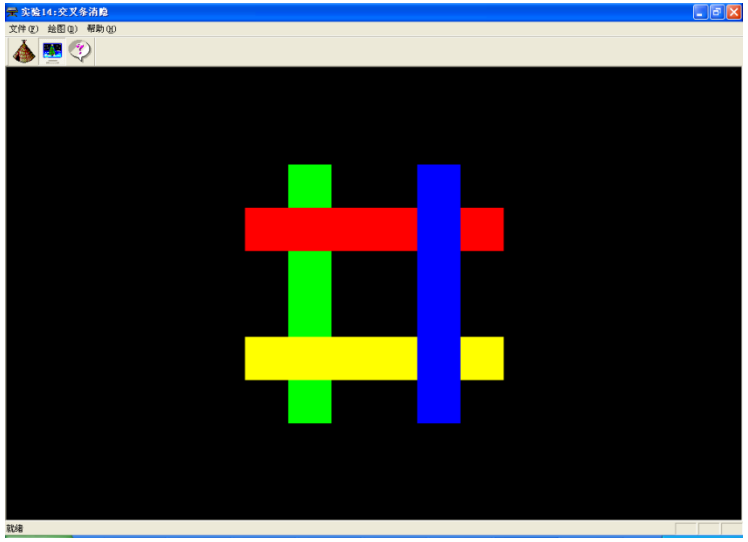


图9-74 叠压条表面模型

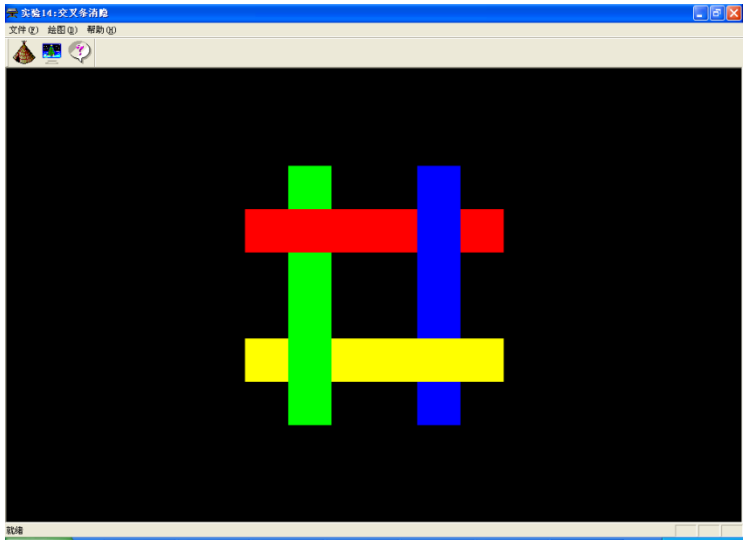


图9-75 交叉条表面模型

9.5 本章小结

1. 本章主要讲述多面体和光滑物体的数学模型建立方法。根据顶点表和面表绘制物体的三维模型。

2. 三维图形的消隐算法分为隐线算法和隐面算法。

- 隐线算法主要针对线框模型进行，只根据表面法矢量和视矢量的夹角就可以进行背面剔除。凸多面体隐线算法也是一种隐面算法，是通过判断表面免的可见性后才绘制面的边界，只不过该方法可用于绘制线框模型，才称为隐线算法。

- 隐面算法中重点讲解了z-buffer算法，该算法是计算机图形学中最主要的消隐算法。一般先对物体的多边形表面进行背面剔除预处理，然后才对可见表面使用基于有效边表算法的z-buffer算法，从像素级角度对物体消隐。

习题 9

1.图9-76所示为正四面体，使用MFC编程实现正四面体的透视投影动态隐线算法。这里的”动态”是指使用键盘方向键或动画按钮可以对正四面体进行任意角度的旋转。

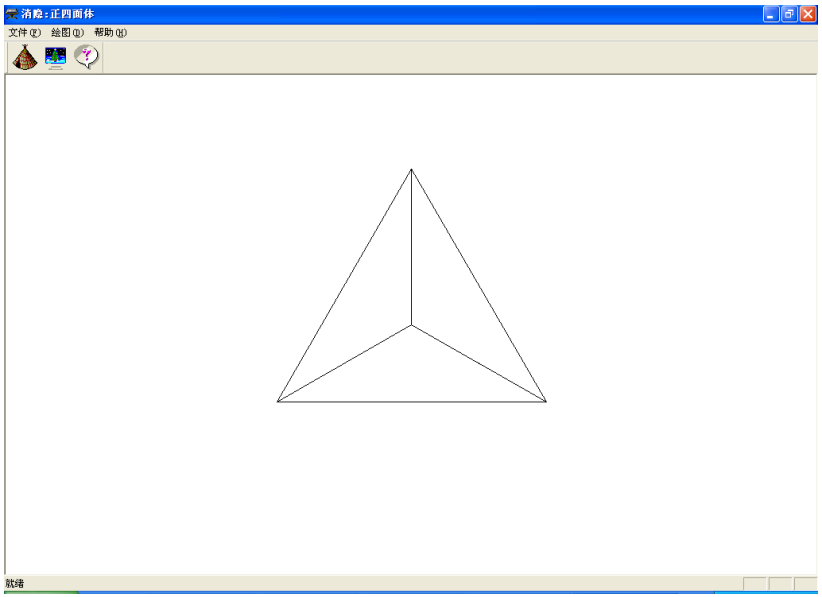


图9-76 正四面体消隐线框模型

2.图9-77所示为正三棱柱线框模型，使用MFC编程实现正三棱柱的透视投影动态隐线算法。

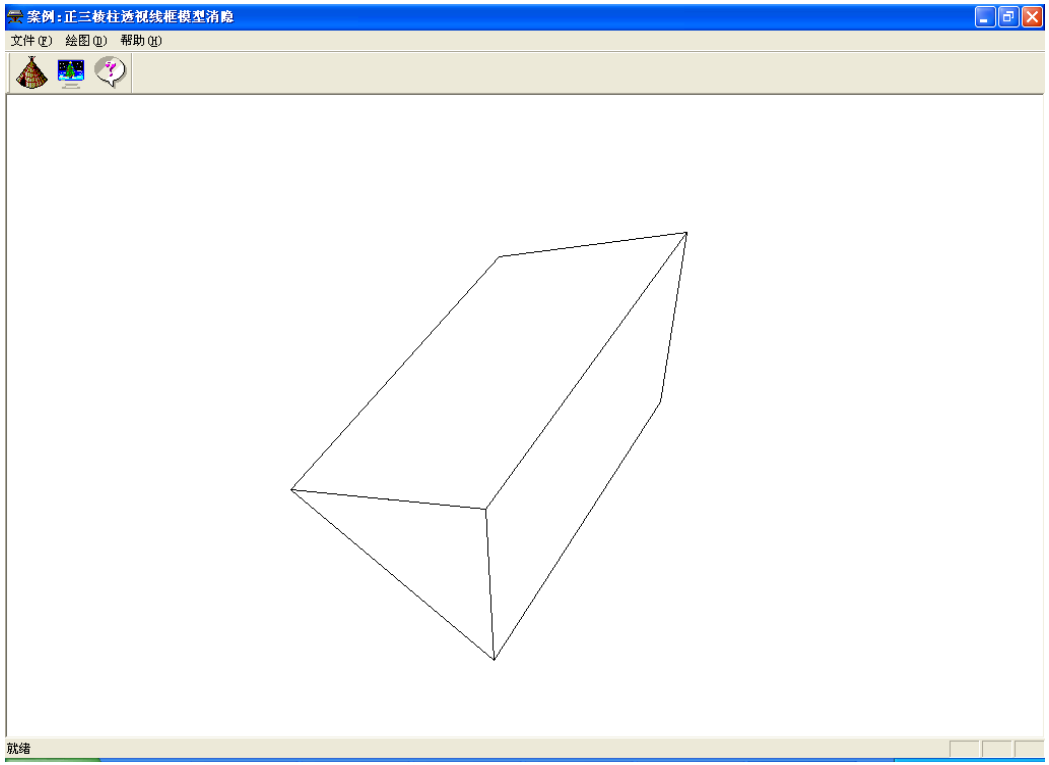
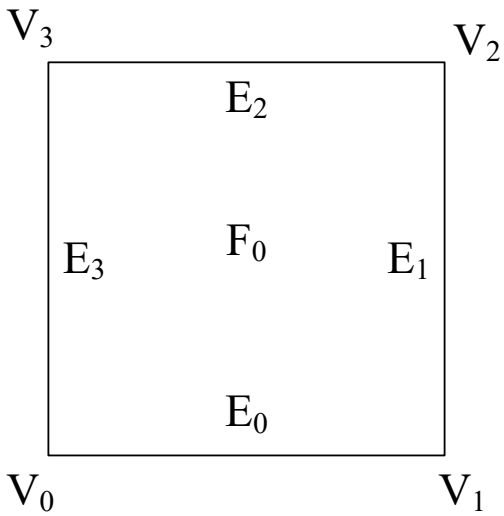


图9-77 正三棱柱消隐线框模型

3.立方体的表面F_0可以通过四个顶点$V_0V_1V_2V_3$来定义，也可以通过四条边$E_0E_1E_2E_3$来定义，如图9-78所示。请编程绘制消隐线为虚线的立方体线框模型透视投影，效果如图9-79所示。

说明：如果仅使用顶点表和面表的双表结构绘制立方体线框模型，每条边被绘制两次。只有借助于边表，使用三表结构才能完成本题。



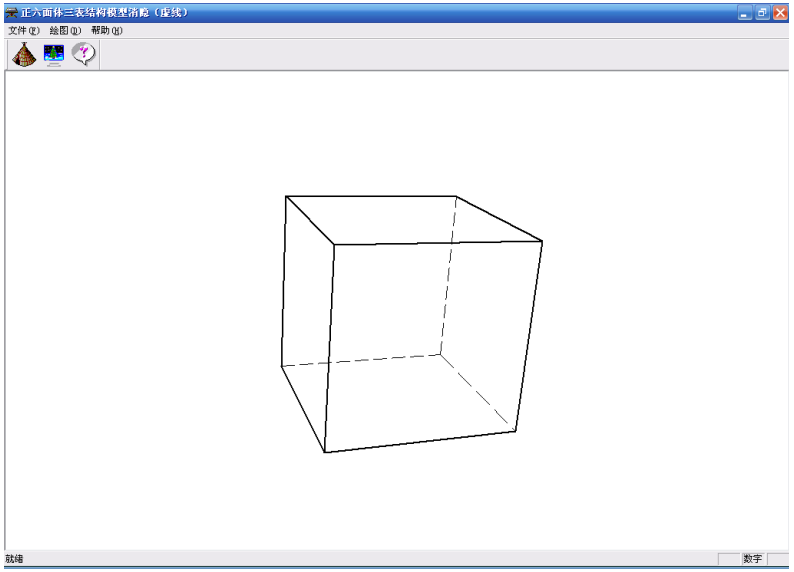


图9-78 立方体表面的定义　图9-79 立方体虚线消隐线框模型

4.如果一个平面在三维坐标系的三个坐标轴上的截距都相等，在第一卦限内的图形如图9-80所示。在八个卦限内的图形可以组成正八面体，如图9-81所示，请使用MFC编程实现正八面体的动态隐线算法。

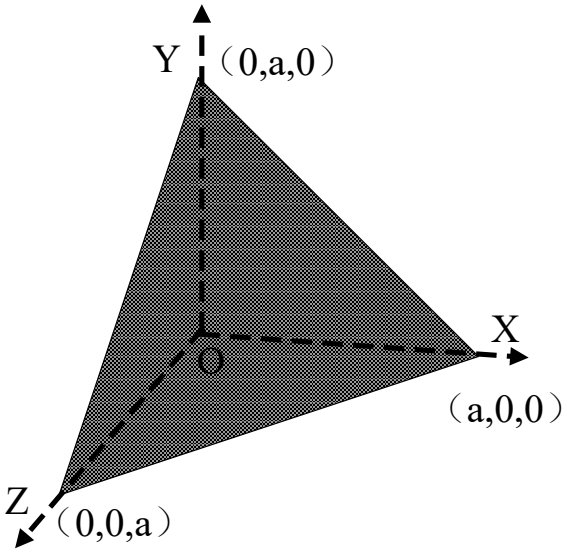


图9-80 平面的截距

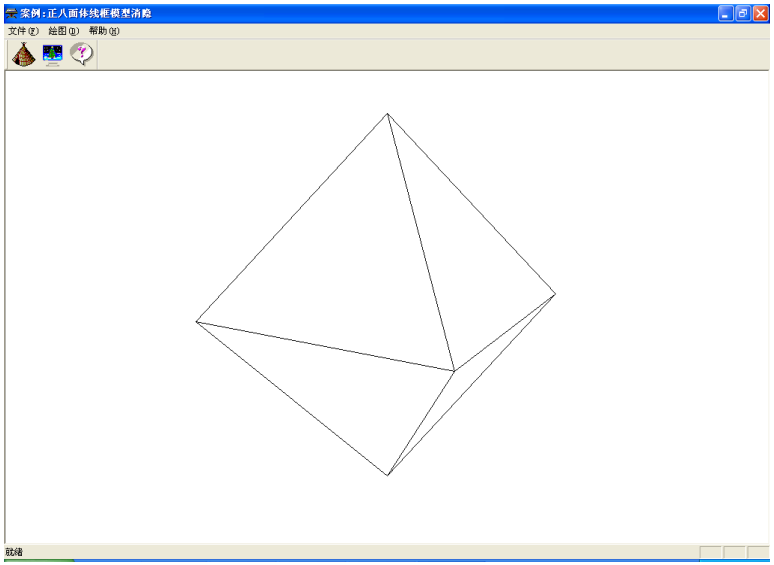


图9-81 正八面体消隐线框模型

5.房屋的三视图如图9-82所示，请以图中的黑点所代表的回转中心为原点建立三维右手坐标系，并使用MFC实现动态隐线算法，绘制如图9-83所示的房屋动态消隐线框模型透视图。

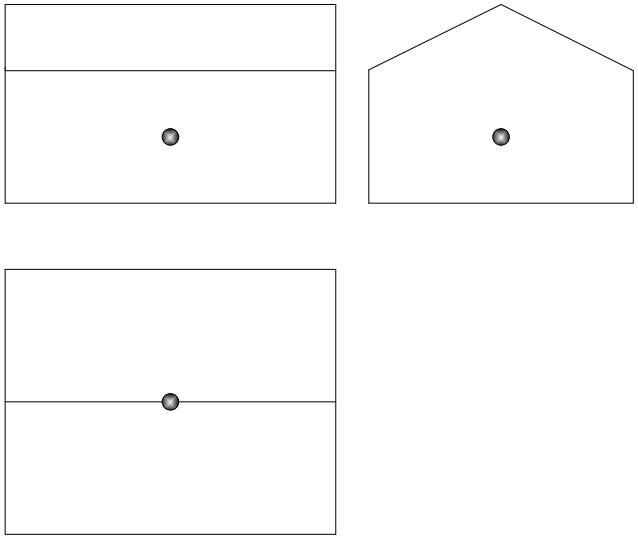


图9-82 房屋三视图

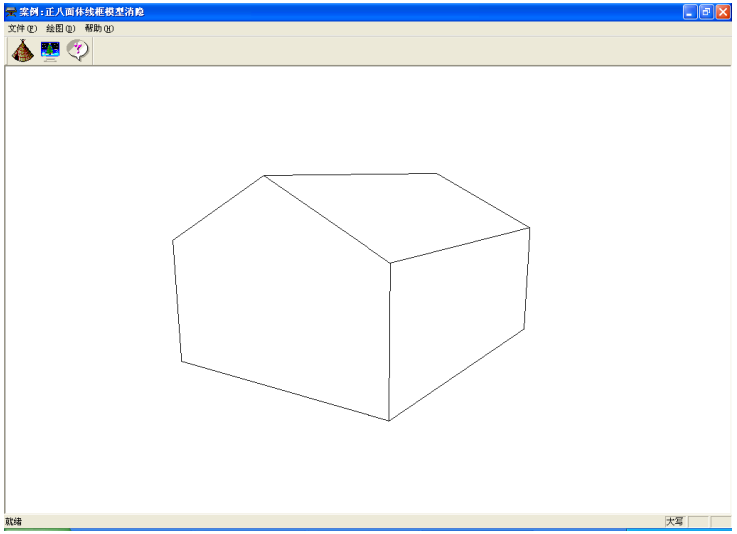


图9-83 房屋消隐线框模型

6.立方体的画家算法也可以这样来实现。立方体共有6个面和8个顶点，每3个面共用一个顶点，分别确定每个顶点的周围的3个面，有8种情况。在立方体旋转的任一角度，计算立方体离视点最近的顶点最远的顶点的z坐标值。先绘制z值大（离视点远）的顶点所对应的3个面，后绘制z值小（离视点近）的顶点所对应的3个面，就可以实现立方体的动态消隐。事实上，立方体属于凸多面体，只绘制离视点最近的顶点所对应的3个面就可以实现消隐，效果如图9-84所示，请编程实现。

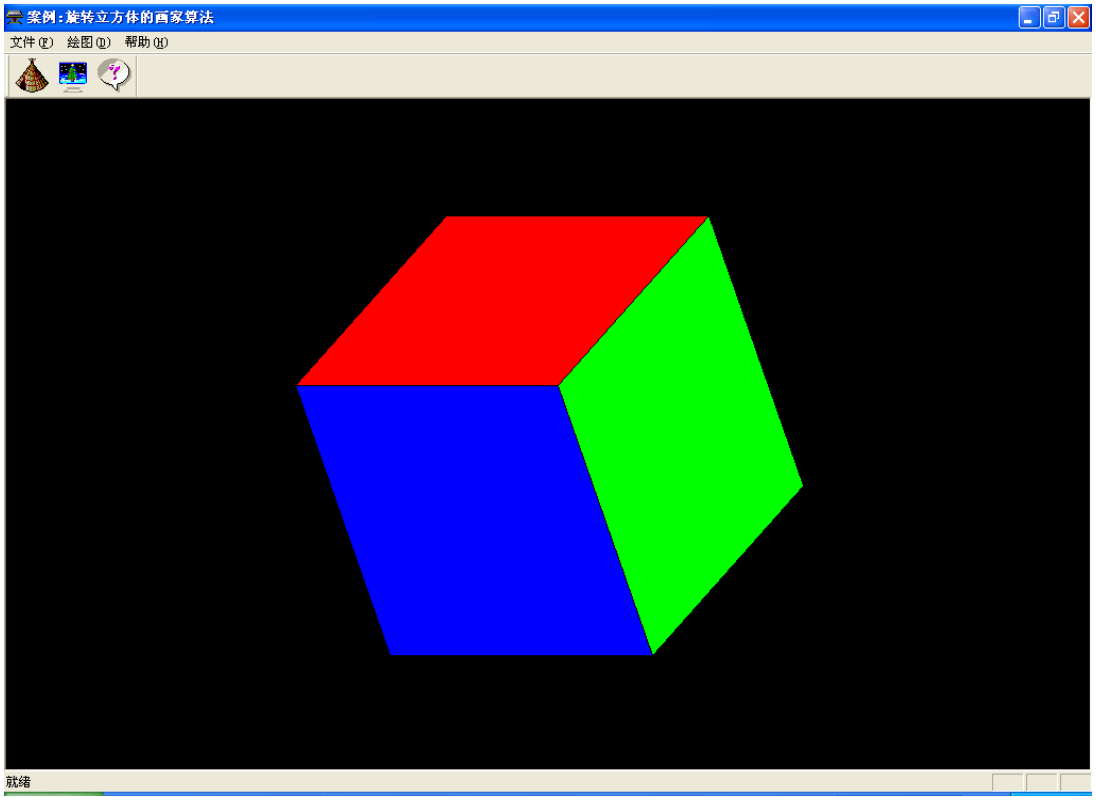


图9-84 立方体表面模型的画家消隐算法

7. z-buffer算法的代表性案例是绘制三个相互交叉的红绿蓝条，如图9-85所示，请使用MFC编程实现。

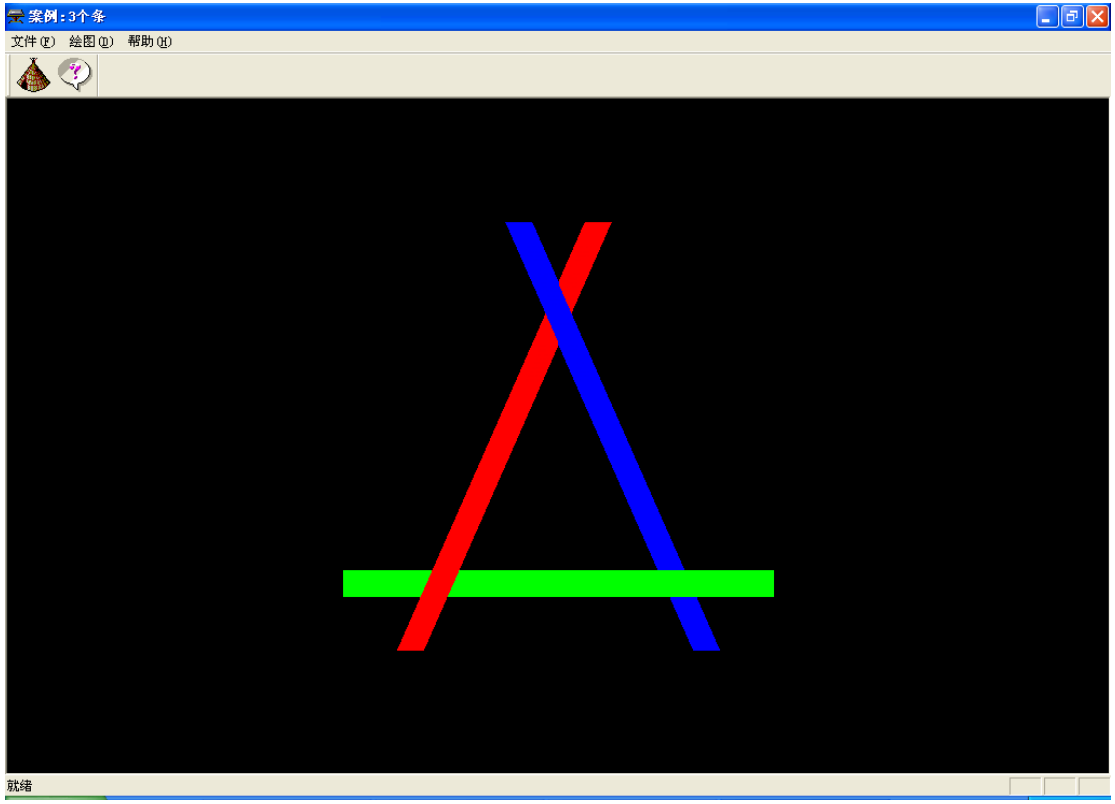
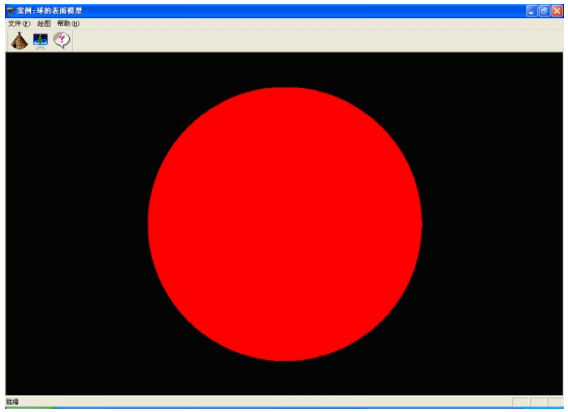


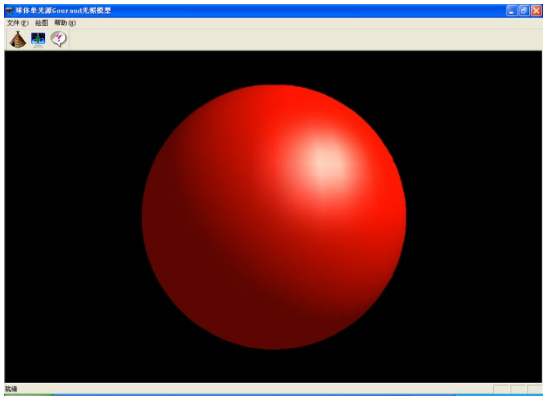
图9-85 三个条

8.先使用三角形网格和四边形网格绘制球的线框模型，然后使用红色填充所有网格单元。请使用MFC编程实现，效果如图9-86（a）所示，要求所填充的球面网格连接处不能出现漏点。

说明：对于球面，如果仅使用单色填充，绘制结果仅仅是一个平面圆，没有三维效果。如果需要在屏幕上展示真正的三维球面，则得借助于光照模型才能实现。在球面的右上方设置白色光源后，球面的光照效果如图9-86（b）所示。



（a）无光照



（b）有光照

图9-86 球的表面模型

9. *建立立方体的三维模型，使用递归算法搭建Menger海绵的生成元，使用斜等测投影绘制画家算法消隐的Menger海绵，如图9-87所示。

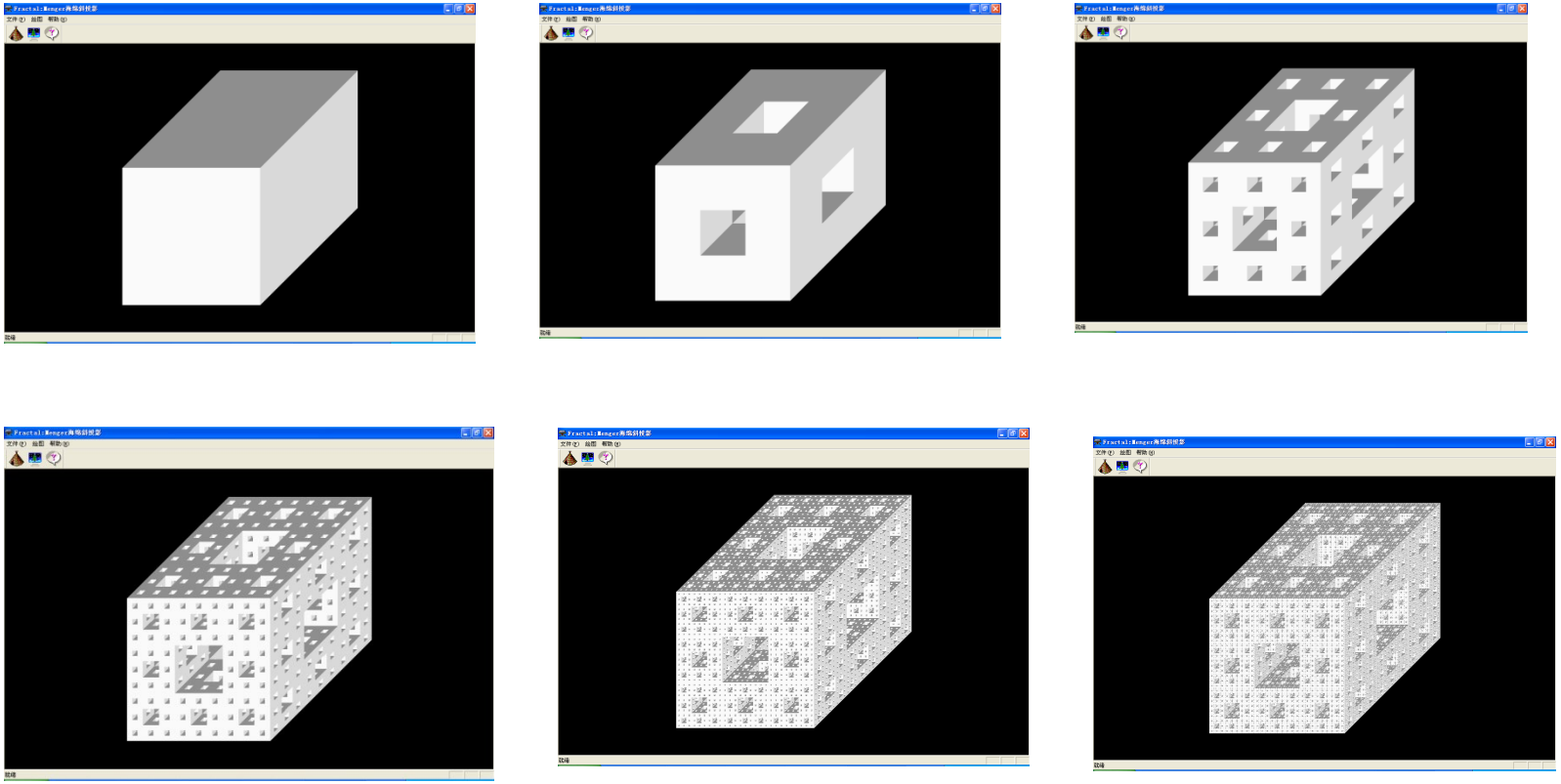


图9-87 斜等测投影绘制的Menger海绵表面模型

10. *使用画家算法编程绘制递归深度为0、1和2的Menger海绵。要求Menger海绵使用正交投影绘制，并且能动态旋转，如图9-88所示。

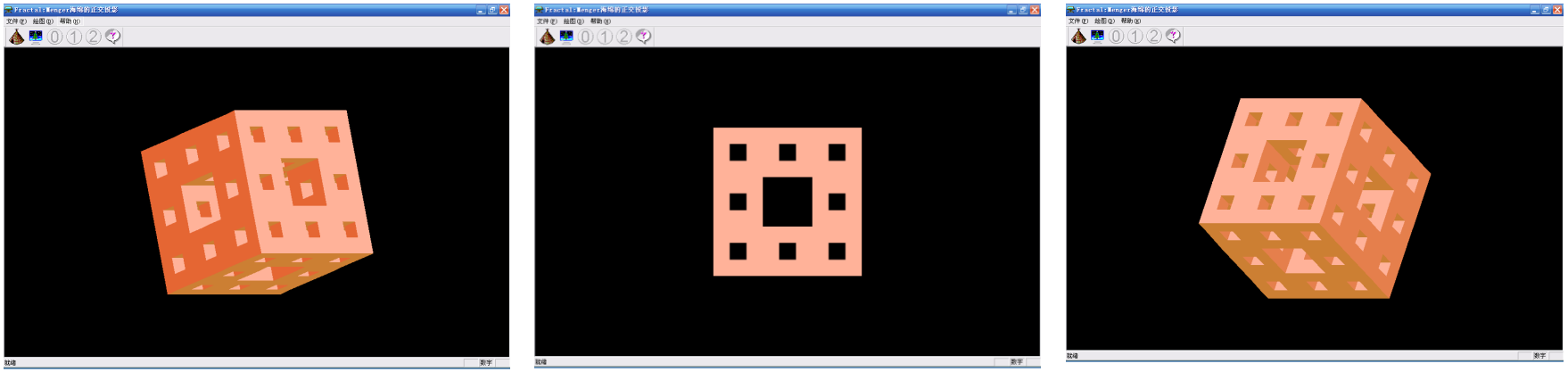


图9-88 Menger海绵表面模型的正交投影

11. *使用画家算法编程绘制递归深度为0、1和2的Menger海绵。要求Menger海绵使用透视投影绘制，并且能动态旋转，如图9-89所示。

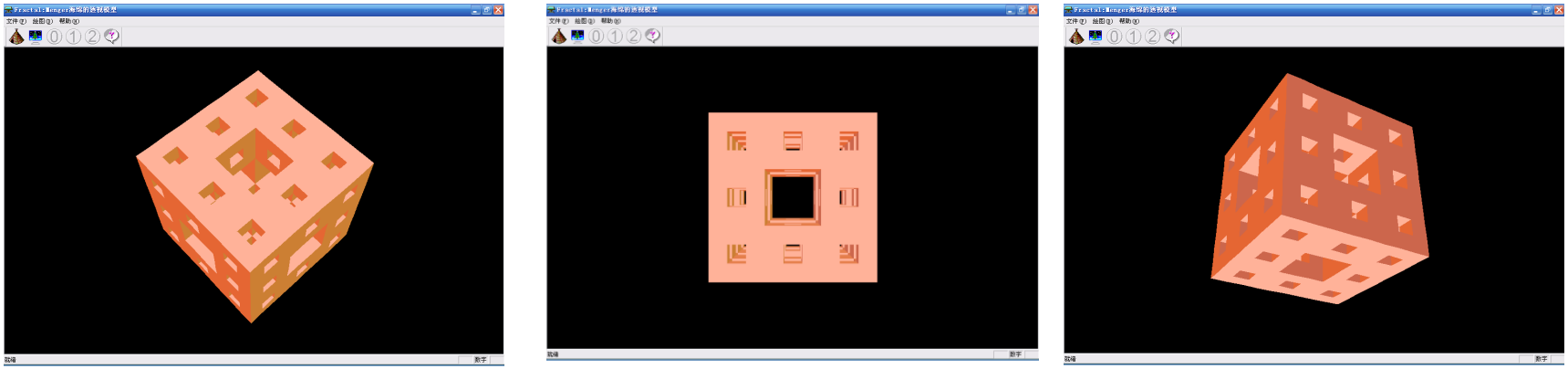


图9-89 Menger海绵表面模型的透视投影

12. *正二十面体有12个顶点，且从每个顶点引出5条边。假设用一把锋利的刀沿着每条边长的1/3处斜截每一个顶点，这样会留下由12个正五边形和20个正六边形组成的球体，如图9-90所示。这个有60个顶点的球体被称为Bucky球，类似于足球。请编写点表和面表绘制如图9-91所示的Bucky球线框图。

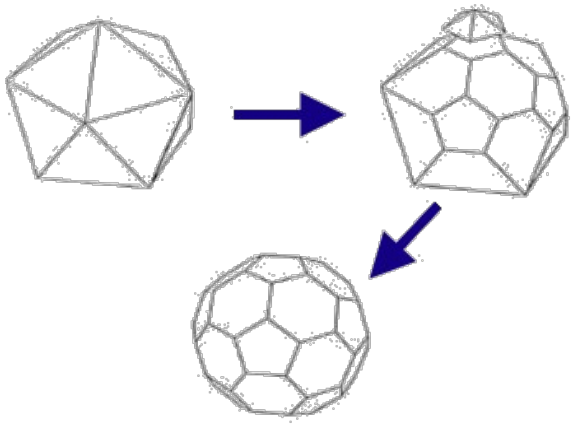


图9-90 斜截正二十面体形成Bucky球

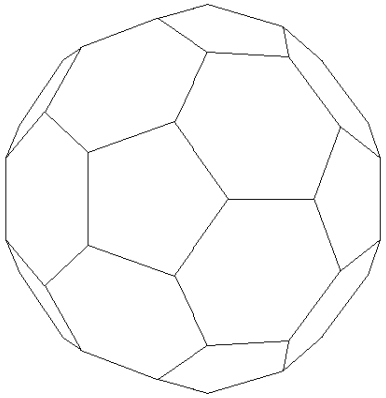


图9-91 Bucky球线框图

13. *Bucky线框图的五边形与六边形表面均为平面。请连接正五边形的中心和正五边形的每个顶角，并将中心点拉至球面上。对正六边形也行类似的操作，得到了足球的线框模型。请绘制图9-92所示的足球线框图。

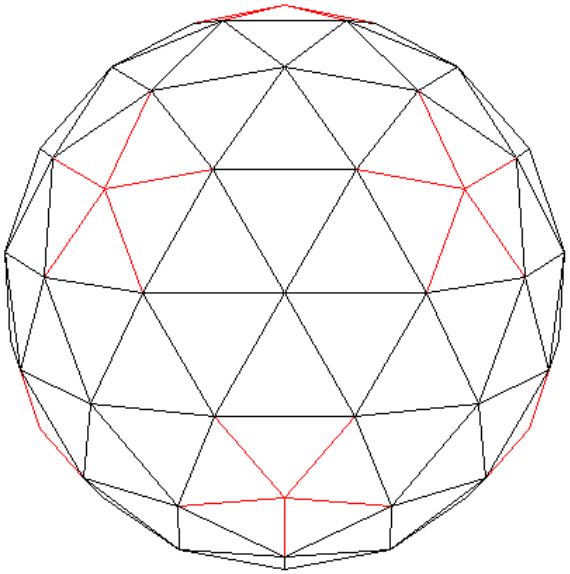


图9-92 足球线框图